scientific reports

Check for updates

OPEN

A resource efficient IoT intrusion detection model using hybrid feature selection for edge computing

Mohd Zain Khan[1], Mahfooz Alam[2], Irfan Alam[3]✉, Mohammad Ubaidullah Bokhari[1], Zubair Ashraf[4], Mohammad Zunnun Khan[5] & Faheem Syeed Masoodi[6]

The Internet of Things (IoT) devices have grown at a very fast rate, which has led to escalated security threats. Most of the current IoT oriented lightweight intrusion detection systems do not maintain a high rate of detection performance with heterogeneous and imbalanced traffic, or the expense of increased computation and memory occurs with high detection rate. To resolve this problem, this paper presents a resource efficient IoT attack detection framework called BGL-RID (Boruta-Greedy LightGBM Resource-Efficient IoT Detection). The framework applies a hybrid feature selection pipeline that integrates both Boruta and Greedy Forward Selection (GFS) to remove unnecessary features and only include the most useful features in the pipeline. The Synthetic Minority Oversampling Technique (SMOTE) is used to deal with the issue of class imbalance. Performance is measured based on accuracy and efficiency ratio, which indicates efficiency between quality of detection and resource consumption. The performance of the proposed BGL-RID model has been tested on benchmark, edge collected and IoT specific datasets namely TONIoT, proxy-labeled Raspberry Pi, CICIDS2018, and CICIoT2023. Experimental results demonstrate strong performance across these datasets. For binary and multiclass classifications, BGL-RID attained 99.72% and 98.94% accuracy on TONIoT dataset respectively. It also attained 99.91%, 99.94%, and 98.88% accuracy on the Raspberry Pi, CICIDS2018, and the IoT-specific CICIoT2023 datasets respectively. Besides having high detection rates, the model also achieves the highest efficiency ratio across different datasets, showing that it is robust and scalable, with minimal computation and memory requirements, suggesting its potential suitability for resource-constrained IoT applications.

Keywords IoT, Greedy Forward Selection, Intrusion Detection System, Hybrid Feature Selection, SMOTE, Binary Classification

The Internet of Things (IoT) refers to a technology that connects devices to the internet. This technology has developed at a very high rate that it has sought to bring products together like wireless sensors, smart cameras, televisions and other smart technologies. Such products are growing and support people to transform the personal and industrial areas by further automating them[1]. Such connectivity also widens the attack area and opens it to remote misuse. IoT deployments facilitate business and safety critical services in which intrusions may discontinue operations and expose sensitive data[2]. Thus, intrusion detection would be a viable need for IoT networks. Meanwhile, a lot of IoT devices operate with minimal CPU capacity, memory, and power, limiting the ability to use security tools with high computation requirements[3]. The focus on these constraints drives the detection methods which are not only accurate but also lightweight and stable to various deployment conditions[4]. The IoT threats are traffic interception, compromising of devices, and Denial of Service (DoS) attacks that affect the availability and the quality of services (QoS)[5,6]. As the IoT networks are interconnected, one weak device can

[1]Department of Computer Science, Aligarh Muslim University, Aligarh 202002, India. [2]Department of MCA, G. L. Bajaj Institute of Technology and Management, Greater Noida 201306, India. [3]Faculty of Computer Studies, Arab Open University-Bahrain, Aali, Kingdom of Bahrain. [4]Department of Computer Science and Artificial Intelligence, College of Computing and Information Technology, University of Bisha, Bisha 67714, Saudi Arabia. [5]Department of Information Systems and Cybersecurity, College of Computing and Information Technology, University of Bisha, Bisha 67714, Saudi Arabia. [6]School of ICT, Bahrain Polytechnic, Manama 973, Bahrain. ✉email: irfan.alam@aou.org.bh

be used as a gateway to the broader compromise[7]. Intrusion detection should be effective and maintain accuracy across changing attacks and other device behaviours[8].

IoT networks comprise various devices, each with distinct hardware capabilities, communication protocols, and an operating system[9]. This inconsistency reduces the reliability of rule-based monitoring and makes it more difficult to maintain attack signatures across diverse environments[10]. Some recent work also demonstrates that, besides intrusion detection, IoT security encompasses secure data auditing, privacy preservation, and distributed learning[11]. Secure data auditing techniques with privacy have been suggested to ensure the integrity of data in the cloud and minimize security risks associated with encrypted data management[12]. Likewise, federated learning has been applied to privacy-preserving classification problems, where the clients retain their data locally but the global model is trained using data from the distributed clients[13]. The studies emphasize the need for secure, lightweight, and privacy-preserving intelligent models in IoT and cloud-based environments[14,15]. There is still a need to explore more resource efficient intrusion detection with reduced feature complexity, particularly for constrained IoT devices.

Machine learning is a viable approach, as it can learn attack patterns from network traffic and system behavior[16,17]. However, the non-homogeneous character of IoT devices such as resource constraints, scalability problems and unreliable network structures is a significant challenge in machine learning based IoT Attack detection[18], as shown in Fig. 1.

Most IoT intrusion datasets have irrelevant and redundant features, which limit the accuracy and increase computation cost[19]. Selection of features is then essential in order to enhance the quality of detection and optimize the resource consumption for deployment on limited devices[20]. It also assists in scalability by dimensional reduction of features whilst maintaining important predictive indicators[21].

Existing IoT intrusion detection approaches primarily target the accuracy of detection, but do not take into account deployment factors simultaneously. A model could have a very high accuracy but be very costly in terms of execution time and memory. These factors are also crucial in resource limited IoT scenarios where devices possess limited processing power, memory and energy capacity. Furthermore, a significant false negative rate can pose a potential problem in security applications as missed attacks can remain undetected. Hence, accuracy alone is not adequate to determine the feasibility of an IDS model for lightweight IoT security. It motivates the need for a resource-aware detection framework, which takes into account the detection performance, false negative rate, execution time and memory usage.

This paper deals with these concerns by presenting a machine learning-based IoT intrusion detection model whose feature selection approach is a hybrid strategy that enhances detection efficiency and performance. It is a combination of Boruta and Greedy Forward Selection (GFS) to select a discriminative feature subset of a compact size which is then used to train a lightweight classifier. This pipeline aims at greater generalization in contrast to the homogeneous environments of the IoT, and decreasing the computations and memory requirements. The major contributions are summarized as follows:

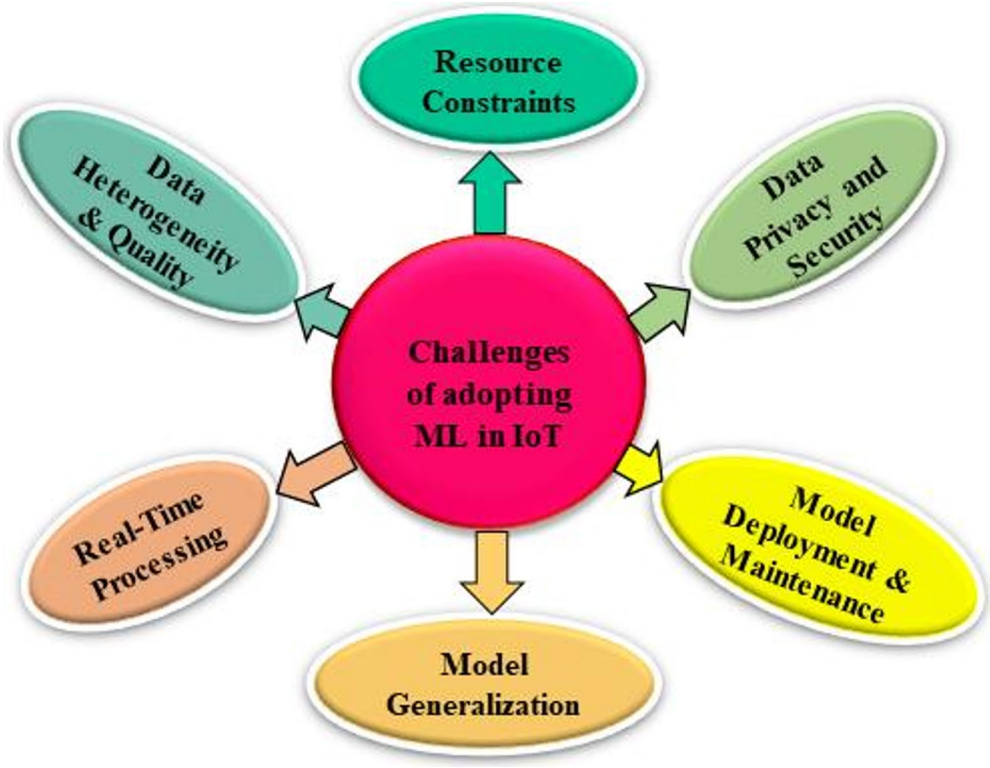


Fig. 1. The major challenges encountered by IoT devices.

- A novel Boruta-Greedy LightGBM for Resource Efficient IoT Detection (BGL-RID) model is proposed for detecting various types of attacks in IoT environments which has the potential to operate under constrained computational resources.
- BGL-RID employs a hybrid feature selection strategy that combines Boruta and greedy algorithms for optimal feature reduction, achieving resource efficiency without compromising classification accuracy.
- BGL-RID model is evaluated on four datasets for comprehensive validation: a custom Raspberry Pi based IoT dataset representing real-world data and the publicly available TONIoT, CICIDS2018, and CICIoT2023 benchmark dataset which provides extensive and diverse attack scenarios.
- A new evaluation metric named efficiency ratio is introduced to measure the trade-off between detection accuracy and the costs of computation, which helps to evaluate the model performance in resource-constrained settings.
- To conduct the performance analysis of the BGL-RID model against baseline models, including accuracy, precision, recall, F1-score, false negative rate (FNR), computational time, memory usage, and an efficiency ratio metric to quantify the trade-off between accuracy and resource consumption.

This study demonstrates that the proposed framework provides a robust, scalable and resource-efficient intrusion detection solution which is a viable solution for resource-aware IoT security applications. The following portions of this article are structured as follows: Sect. "Related work" examines the pertinent related work by various researchers in IoT attack detection using machine learning techniques. Section "Proposed model" presents the proposed methodology that describes each step followed and explains the techniques utilized. Section "Results and discussion" covers the results and discusses the experimental findings. Finally, Sect. "Conclusion" concludes the paper and discusses the future scope of this work.

Related work

A literature review has been performed based on a structured search, screening and selection workflow, under an established research management framework[22]. Various scholars have examined intrusion and attack identification through machine learning methods in IoT environments[23] as illustrated in Table 1. Saheed et al.[24] discussed their knowledge on IDS that applies machine learning to recognize attacks on IoT networks and focus on the role of dimension reduction by use of PCA. Gaber et al.[25] concentrated on effective machine learning-based IDS (ML-IDS) in the detection of injection attacks in a smart IoT. It emphasizes the need to select the eight features that are important in order to have high accuracy with constant removal and RFE. Rahman et al.[26] addressed their perception of combination of feature selection techniques with machine learning to boost accuracy and efficiency. The study primarily addresses impersonation attacks in IoT environments that employ wrapper-based feature selection with SVM, C4.5, and NB to generate an optimized feature set and an ANN is used for classification of impersonation attacks and normal traffic. Shukla et al.[27] introduced a distributed ensemble method designed to detect IoT traffic-based Distributed DoS (DDoS) attacks in near real-time. Computational overhead is minimized by distributing pre-processing and classification activities among several working nodes of storm clusters. Li et al.[28] explain a detailed comparison of feature selection and extraction for

Ref.	Year	Methods	Contribution	Limitations
[24]	2022	XGBoost, KNN, CatBoost, QDA, SVM, and NB	Using PCA to reduce dimensionality and the Min-Max normalization technique to guarantee that all feature values are on the same scale	The suggested framework is evaluated using a single dataset, which is insufficient to measure the generalizability of the model.
[25].	2022	DT, RF and SVM	ML-based IDS for injection attack detection in smart city applications, eight features are selected using constant removal and RFE selection technique.	Features are selected to detect injection attacks only. This imposes limitations on applicability to detect different kinds of attacks.
[26]	2021	SAE, SVM, DT, NB, ANN	ML-IDS that employs wrapper-based feature selection with SVM, C4.5, and NB to generate an optimized feature set. An ANN classifier then leverages these features to effectively distinguish impersonation attacks from normal traffic.	The approach is cloud-dependent with higher CPU time and is limited to impersonation attacks which require extension to other attack types.
[27]	2024	DNN, DRF, GBM, GLM, XGBoost.	Leverages Apache Storm and ensemble learning for high-performance DDoS attack detection which aims to strengthen IoT network security.	Fails to address the computational overhead of the model to deploy in resource constraint environment.
[28]	2024	RF, KNN, NB, MLP	It provides data-driven insights to guide optimal feature reduction choices for efficient and effective IoT attack detection.	Lacks a comparison on modern IoT datasets and class imbalance is not handled.
[29]	2025	RF, LR, NB	It enhances detection accuracy while reducing computational overhead which makes it suitable for IoT environments.	Reliance on outdated datasets such as KDD limits their relevance to current DDoS attack patterns.
[32]	2024	DT, RF, XGBoost	Unified framework for detecting both DNS and ARP spoofing. Random Forest achieves high accuracy.	Performance of models can be improved using advanced feature selection techniques.
[33].	2022	RF, XGBoost	Proposed approach is evaluated and validated using latest dataset which consists of the latest attack patterns with XGBoost showing highest accuracy.	The model can be improved using advanced feature selection techniques.
[34]	2023	SVM, CNN, GRU, NB, XGBoost	Using real-world data with advanced feature selection and SMA optimization to address overfitting and underfitting.	Computational overhead is not addressed to measure the suitability of model to deploy in resource constraint IoT environment.
[35]	2022	Bagging, Multilayer Perception, J48 and IBk (Instance-Based k)	A hybrid feature selection method is utilized which combines Information Gain and Gain Ratio.	Suffer from a high FNR as current feature selection methods fail to fully eliminate irrelevant or redundant features.

Table 1. A summary of state-of-the-art work.

IoT NIDS (Network Intrusion Detection System) which highlights their respective strengths and weaknesses in terms of performance, computational efficiency and impact on various machine learning models across different feature quantities. Nawaz et al.[29] addressed a machine learning framework for IoT DDoS detection using random forest with extra trees feature selection to achieve high accuracy and minimize computational overhead. Saied et al.[30] provided a comparative analysis of boosting-based methods for detecting IoT botnets and demonstrated that boosting models can deliver high-quality multiclass detection with consideration of time efficiency, which is a valuable consideration in resource-limited IoT settings. Saied and Guirguis[31] proposed an explainable AI architecture to detect IoT botnets and reported the combination of interpretation methods, including rule extraction, LIME, and SHAP, to enhance transparency and trust in the detection process. Vajrobol et al.[32] explain an efficient machine learning-based method to detect DNS and ARP spoofing attacks in IoT environments with high accuracy. It focuses on the detection of both types of attack using a comprehensive set of features within a unified framework. Ismail et al.[33] addressed their work on machine learning-based approach for predicting DDoS attacks. They have emphasized the use of latest datasets to reflect current attack patterns. Ahmad et al.[34] focused on a big data analytics-based approach for DDoS detection in IoT environments using an optimized ensemble framework. To address overfitting and underfitting problems, they have employed SMA optimization. The need for strong security solutions has been emphasized because of the growing IoT vulnerabilities and the evolving cyber threats with new patterns. Albulayhi et al.[35] presented an IDS approach for IoT that tackles effectively the challenges of high-dimensional and diverse data to improve IDS performance by using a hybrid feature selection approach. Saied et al.[36] made a comparative analysis of six boosting-based machine learning algorithms in intrusion detection of IoT networks and tested them in terms of multiclass classification and time-related performance measures. They also examined the cost of computation to determine lightweight boosting models that can be used in the resource-constrained IoT. Anish et al.[37] suggested a hybrid technique for feature extraction in IDS using a combination of gain ratio and correlation-based analysis. It removes irrelevant features, enhances detection accuracy and decreases the computational complexity in ML-based NIDS. For ICN-IoT caching, Asmat et al.[38] presented an ECCC framework based on PPO to increase the cache hit ratio, decrease server load, and optimize energy usage by making adaptive decisions about the cache. Ullah et al.[39] presented a lightweight knowledge-distillation model for multimedia anomaly detection and integrity verification in CIoT environments, named MAID-KD. For edge-based IoT security, they have developed a method that combines transformer teacher and CNN-LSTM student models to achieve high accuracy with low model size and inference latency. Uthansakul et al.[40] proposed a QoE-aware self-tuning method for LTE resource allocation. They applied ANN to predict the QoE and a particle genetic algorithm to optimize the service priority factors, thereby making optimal resource allocation under limited network conditions.

Table 2 summarizes the key differences between the BGL-RID model and the current feature selection-based IoT IDS pipelines. Despite the existence of many studies, which can be used to detect intrusion in IoT, there are several research gaps identified. First, most of the current IoT IDS methods are tested with only one dataset, making it difficult to support the generalization of the solutions across different traffic patterns and attack scenarios. Second, most studies only focus on the accuracy of detection and do not jointly take into account runtime, memory usage, and false negative rate. This makes it challenging to determine the feasibility of using a model with resource-limited IoT devices. Third, some methods employ one-stage feature selection or fixed feature ranking, which can either miss out on informative features at an early stage or keep redundant features. Fourth, class imbalance is not always effectively addressed, which could lead to higher missed attack rates. The BGL-RID model overcomes these drawbacks by proposing Boruta and GFS for compact and relevant feature selection, SMOTE for class balancing, and LightGBM for efficient classification, and checking the performance of the proposed model on various datasets based on accuracy, FNR, execution time, memory usage, and efficiency ratio.

Proposed model

This section presents the proposed methodology of BGL-RID Model in detail. It is an IoT-based Intrusion Detection System (IDS) pipeline with a LightGBM classifier incorporated with hybrid feature selection consisting of Boruta and Greedy Forward Selection to yield a feature subset that can be compact and stable. It also reports resource- aware testing with runtime and memory cost as well as detection quality.

Aspect	Existing IoT IDS pipelines	Proposed BGL-RID framework
Feature selection design	Often use a single filter, wrapper, embedded method, or fixed top-k ranking.	Uses a two-stage Boruta and Greedy Forward Selection strategy.
Feature relevance	May remove useful features early when only ranking-based methods are used.	Boruta first identifies all relevant features using shadow-feature comparison.
Feature compactness	Selected features may remain redundant or oversized.	Greedy Forward Selection further keeps only features that improve validation performance.
Class imbalance handling	Class imbalance is sometimes not directly integrated into the full IDS pipeline.	SMOTE is integrated to reduce majority-class bias and improve attack detection.
Classifier design	Often focus mainly on accuracy.	LightGBM is used to balance detection accuracy, runtime, and memory usage.
Resource-awareness	Computational cost is often treated as a secondary metric.	Runtime, memory usage, FNR, and efficiency ratio are considered together.

Table 2. Comparison of the BGL-RID with existing IoT IDS pipelines.

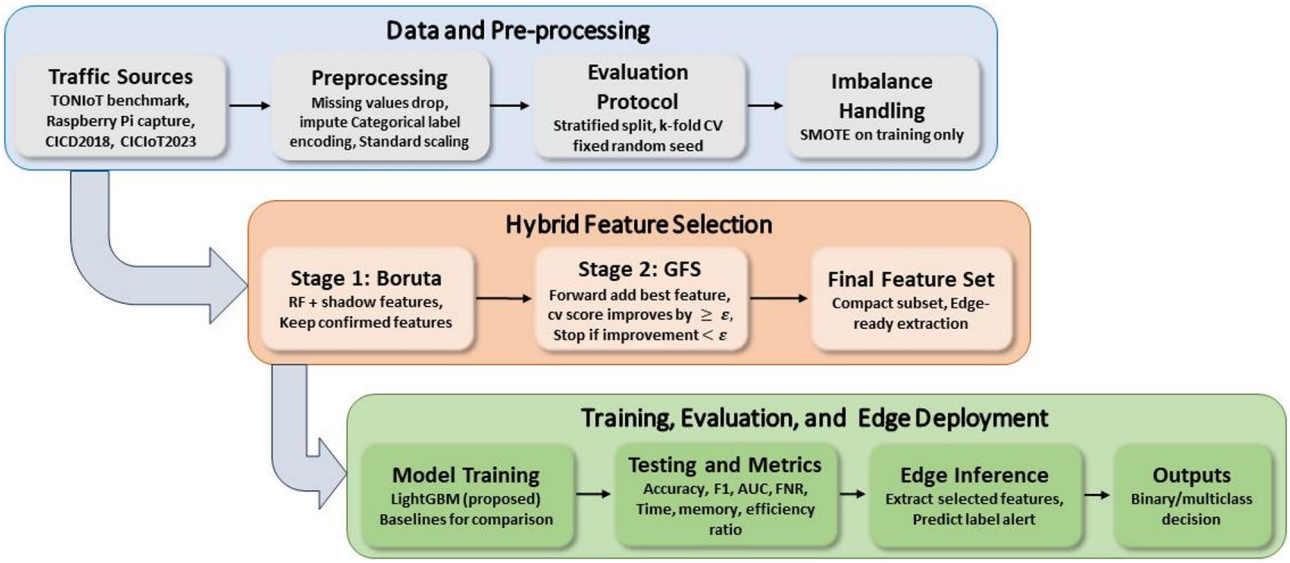


Fig. 2. System architecture depicting dataflow.

Notation	Meaning	Notation	Meaning
D	Raw dataset	C	Classifier
D_{clean}	Preprocessed data	$X\left[G'\right]$	Feature matrix with the selected features
F_{cat}	Set of categorical features	k	Number of cross-validation folds
X	Feature matrix	N	Number of samples
Y	Target labels	M	Number of Classes
$X_{\text{smote}}, y_{\text{smote}}$	Balanced dataset using SMOTE	d	Original features
F_{Boruta}	Features selected through Boruta Algorithm	$f_t\,(x)$	A newly added tree at iteration t
G_{greedy}	Features selected through Greedy Algorithm	η	Learning rate
X_{final}	Final selected features after feature selection	x	Feature vector
C_k	Classifiers	w	Weight vector
T	Number of trees	b	Bias term
$\Delta I_j^{(t)}$	Impurity reduction contributed by the feature X_j in tree t	∇L	Gradient of the loss
y	True label	p_k	Class probability
$\widehat{y}$	Final predicted labels	$K\,(x_i, x)$	Kernel function
$\widehat{y_i}$	Predicted label for the ith instance in the test set	ϵ	Minimum gain threshold
		CV	Cross Validation folds

Table 3. Description of Notations.

The main goal of the study is to design a resource efficient model where IoT attack detection has a FNR as low as possible, execution time as minimal as possible, memory usage as minimal as possible and detection accuracy maximized. There is also an assessment of this model using the primary and secondary datasets. We have used preprocessing methods to make sure that all outliers that may have an influence on the performance of the model are eliminated. To deal with the issue of imbalance in classes, the SMOTE technique is used, which assists in creating synthetic information of the minority class to balance the data. The above preprocessing methods aid in preparing the data in a form that is trainable and ensure that overfitting and underfitting problems are reduced. After that, feature selection is applied to select the most pertinent features. Boruta is first applied to select the statistically significant features, then GFS technique is applied to further refine and select the most relevant features that highly contribute to the prediction of IoT attacks. The hybrid method guarantees the selection of the most meaningful features that increase the detection performance without compromising computational performance, which is an imperative need within resource-constrained IoT settings, and therefore, making intrusion detection scalable and real-time with less processing overhead and enhanced system reliability. Figure 2 shows a detailed system architecture which show the dataflow, the stages of feature selection used and the deployment of the classifier. The description of notation used is presented in Table 3.

We have employed a fine-tuned LightGBM classifier as depicted in Table 4 to ensure a lightweight solution for prediction of IoT attacks with minimum execution time. These parameters are chosen to enhance accuracy and ensure resource efficiency such that.

- **num_leaves and max_depth** represent limiting tree depth and preventing overfitting by controlling model complexity.
- **feature_fraction and bagging_fraction** represent random sampling of features to improve generalization and computational efficiency.
- **min_data_in_leaf** represents that the tree does not create small splits which further avoids overfitting.
- **Goss (Gradient-based One-Side Sampling)** represents a boosting method that optimizes training time without compromising model performance.
- **random_state** represents the set for reproducibility and consistency across experiments.

In this study, the efficiency ratio measures the tradeoff of detection performance and resource optimization. It evaluates attack detection performance without compromising its computational performance which ensures

Parameter	Description	Value
LightGBM		
n_estimators	Number of trees	200
num_leaves	Controls complexity (number of leaves)	31
max_depth	Limits tree depth to avoid overfitting	6
feature_fraction	Randomly selects a fraction of features	0.7
bagging_fraction	Randomly selects a fraction of data	0.7
min_data_in_leaf	Minimum data in a leaf to prevent overfitting	50
boosting_type	Boosting method used	goss
random_state	Random seed for reproducibility	42
CatBoost		
iterations	Number of boosting iterations	200
depth	Tree depth	6
learning_rate	Learning rate	0.10
subsample	Fraction of samples per iteration	0.7
loss_function	Loss function	Logloss
verbose	Controls training log output	0
random_seed	Random seed for reproducibility	42
Random Forest/Extra Trees		
n_estimators	Number of trees	200
max_depth	Limits depth to reduce complexity	12
max_features	Features considered per split	sqrt
min_samples_leaf	Minimum samples per leaf	2
n_jobs	Parallel CPU usage	-1
random_state	Random seed for reproducibility	42
Gradient Boosting		
n_estimators	Number of boosting stages	200
learning_rate	Learning rate	0.10
max_depth	Depth of individual estimators	3
subsample	Fraction of samples per stage	0.7
random_state	Random seed for reproducibility	42
SVM		
kernel	Kernel type	rbf
C	Regularization strength	1.0
gamma	Kernel coefficient	scale
probability	Probability estimates (adds extra cost)	False
random_state	Random seed for reproducibility	42
Logistic Regression		
solver	Optimization solver	liblinear
C	Inverse regularization strength	1.0
max_iter	Maximum iterations	1000
random_state	Random seed for reproducibility	42

Table 4. Description of the parameters considered in the BGL-RID model.

smooth deployment in resource constrained IoT environment. The suggested model is experimentally validated on four datasets to measure its generalizability in real-world IoT scenarios as illustrated in Fig. 3. For the ToNIoT dataset binary and multiclass classification tasks were conducted to evaluate the model's capability in distinguishing between various attack categories and normal network traffic.

In contrast, for the custom-generated Raspberry Pi dataset, binary classification was performed to effectively discriminate between benign and malicious network behavior in an actual edge-based IoT environment. The consistent results across different datasets affirm the reliability and efficiency of the BGL-RID framework in detecting diverse IoT attacks under a constrained computational environment.

Input: Raw dataset D with labeled instances

Result: Final predicted labels $\hat{y} = \{\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_n\}$, Evaluation metrics

Step 1: Data Cleaning: Removing instances with missing values to obtain: $D_{\text{clean}} = \{x_i \in D \mid \forall j, x_{ij} \neq null\}$

Step 2: Encoding Categorical Features: $X_{encoded}\,[f] = LabelEncoder(X[f]) \forall f \in F_{cat}$

Step 3: Feature Scaling: Apply standard scaling to normalize the features: $X_{Scaled} = \frac{X - mean\,(X)}{std\,(X)}$

Step 4: Hybrid Feature Selection: Boruta Feature Selection- $F_{Boruta} \subseteq F$ and GFS- $G_{greedy} \subseteq F_{Boruta}$

The goal is to maximize the classification performance.

$$G_{greedy} = arg \max_{G' \subseteq F_{Boruta}} Score(G')$$

Where the score function is defined based on cross-validation accuracy

$$Score(G') = \frac{1}{k}\sum_{i=1}^{k} accuracy(C, X[G'])$$

The final set of features used for model training is $X_{\text{final}} \subseteq X_{Greedy} \subseteq X_{\text{Boruta}}$

Step 5: Train-Test Split: Split the dataset (X_{final}, y) into training and testing sets.

$$(X_{train}, X_{test}), (Y_{train}, Y_{test}) = train_test_split(X_{final}, y, test_{size} = 0.2)$$

Step 6: Class Imbalance Handling Using SMOTE: $(X_{\text{smote}}, y_{\text{smote}}) = SMOTE(X_{train}, y_{train})$

Step 7: Model Training and Evaluation: For each classifier $C_k \in$ {Extra Trees, Random Forest, SVM, ... }

$$\hat{y}_i = C_k(X_{Smote}), \quad \forall i \in \{1,2,3, \ldots\ldots, n_{test}\}$$

Algorithm. Proposed BGL-RID Model.

Dataset

In this study, we have utilized four datasets which include Raspberry Pi Dataset which is a primary dataset and the secondary datasets are TONIoT, CICIDS2018, and CICIoT2023.

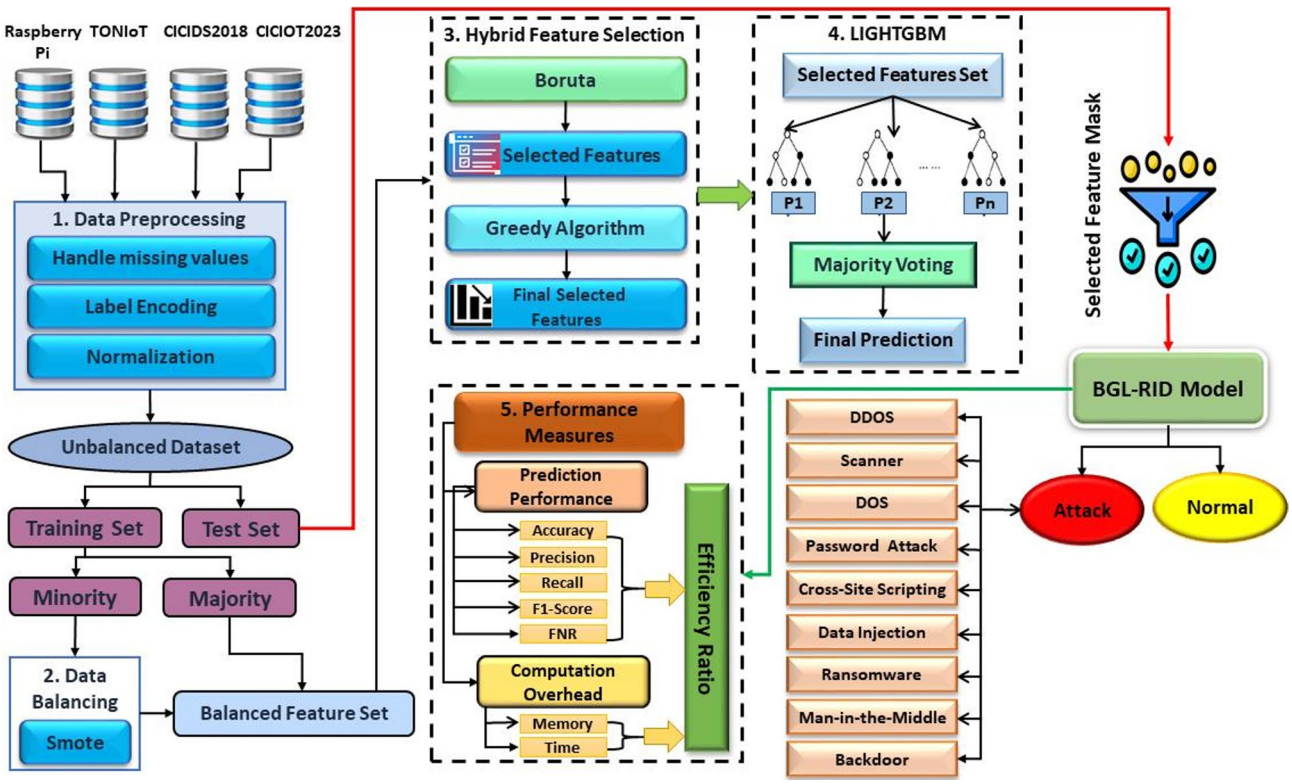


Fig. 3. Proposed architecture of BGL-RID Model.

Raspberry Pi dataset

Raspberry Pi data was collected in a controlled local network. Raspberry Pi device and the laptop were connected to same Wi-Fi network before traffic collection, to enable direct communication between traffic source and target device. RealVNC was used to remotely access the Raspberry Pi. In the experiment, the device used was a Raspberry Pi 4 Computer Model B with 4 GB RAM, 64 bit quad-core ARM Cortex-A72 processor, 4 GB LPDDR4 RAM, Gigabit Ethernet, 802.11b/g/n/ac Wi-Fi, Bluetooth 5.0, and a 5 V/3A USB-C power supply. The Raspberry Pi's wireless interface was used to capture packet traffic using tcpdump and save it in PCAP format. The DoS traffic scenario was controlled and generated from the laptop against the Raspberry Pi in the same local network and traffic was recorded. The captured PCAP file was then processed with TShark to view the fields available in the PCAP file and to extract selected fields of the packets, including the frame number, source IP, destination IP, frame length, protocol/header fields, and converted to CSV. TShark is the terminal-based version of Wireshark. This dataset contains 771 features per record and spans around 435 MB of traffic data that is collected across various networking layers and protocols. A single network packet is represented in each row with timestamped fields and extracted metadata from real-world traffic that includes link, IP, transport and application-level attributes. This dataset does not contain explicit labels that indicate attack or benign behavior but the presence of diverse protocol attributes and traffic metadata makes it ideal for unsupervised or weakly supervised IDS. Hence, the benign and malicious annotations of the Raspberry Pi dataset are not actual attack labels, but proxy labels obtained by applying the Isolation Forest anomaly detection algorithm. Isolation Forest was trained exclusively on the training set, and then applied to the test set to generate proxy labels. This dataset provides opportunities for feature selection and pattern recognition because of high dimension and field richness. The extracted features can be broadly grouped into the following categories:

- Temporal and Frame Metadata: Includes absolute and relative timestamps (frame.time_epoch, frame.time_relative), frame length and interface encapsulation types.
- Network and Transport Layer Features: Contains IP-level fields (ip.src, ip.dst, ip.ttl), TCP/UDP ports, flags, sequence/acknowledgment numbers, window size, and retransmission indicators (tcp.analysis.retransmission).
- Application Layer Protocols: Encompasses DNS (dns.qry.name, dns. flags, etc.), mDNS, NBNS, DHCP and HTTP-related fields extracted from packet payloads.
- Protocol Analysis Flags and Indicators: Includes expert flags and diagnostics such as checksum validity, out-of-order detection, duplicate ACKs and malformed packet markers.

The proxy labels of the Raspberry Pi traffic were generated using Isolation Forest since the packet-level ground-truth labels could not be obtained in all records. This model generates a score for every sample. A score greater than 0 means an outlier in the model is fitted. Proxy malicious labels are made in case of an anomaly score of 0 or more. Otherwise, proxy benign labels are used. Figure 4 indicates the distribution of anomaly score and the labeling threshold. The use of threshold 0 is due to the fact that Isolation forest sets a standard level of a decision border at decision function (x) = 0, where negative numbers denote outliers and positive numbers denote inliers.

An integrity check at a feature level was done to minimize the confusion in proxy labeling. There is an apparent change in TCP sequence and acknowledgment related feature in proxy malicious samples, absent in proxy benign samples. These areas mirror the behavior at the session level and are likely to vary in case of abnormal traffic conditions. The deviations of the proxy classes are reported in Fig. 5, which confirms the existence of proxy malicious samples that are not random outliers.

Additionally, a manual inspection of representative proxy benign and proxy malicious samples was performed, looking at packet-level fields and TCP-related packet attributes such as sequence number, acknowledgement number, packet length and timing-related behavior. This traffic-level inspection further confirmed the results from the Isolation Forest traffic labels were valid. Table 5 summarizes the traffic-level interpretation used to support the proxy labels generated by the Isolation Forest, based on the anomaly score distribution shown in Fig. 4 and the distribution of the TCP-related features shown in Fig. 5.

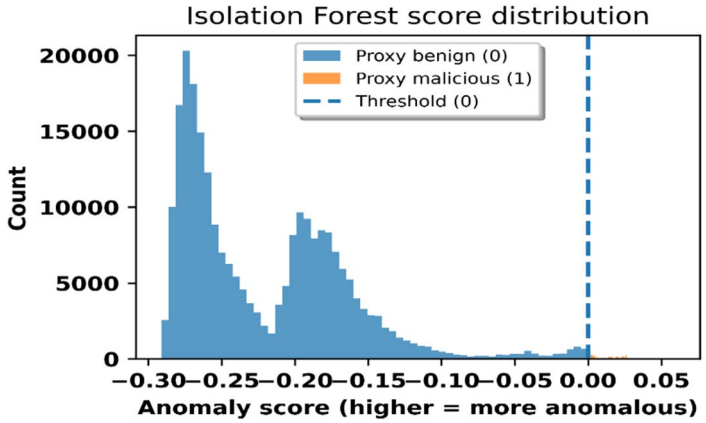


Fig. 4. Isolation Forest score distribution (threshold = 0).

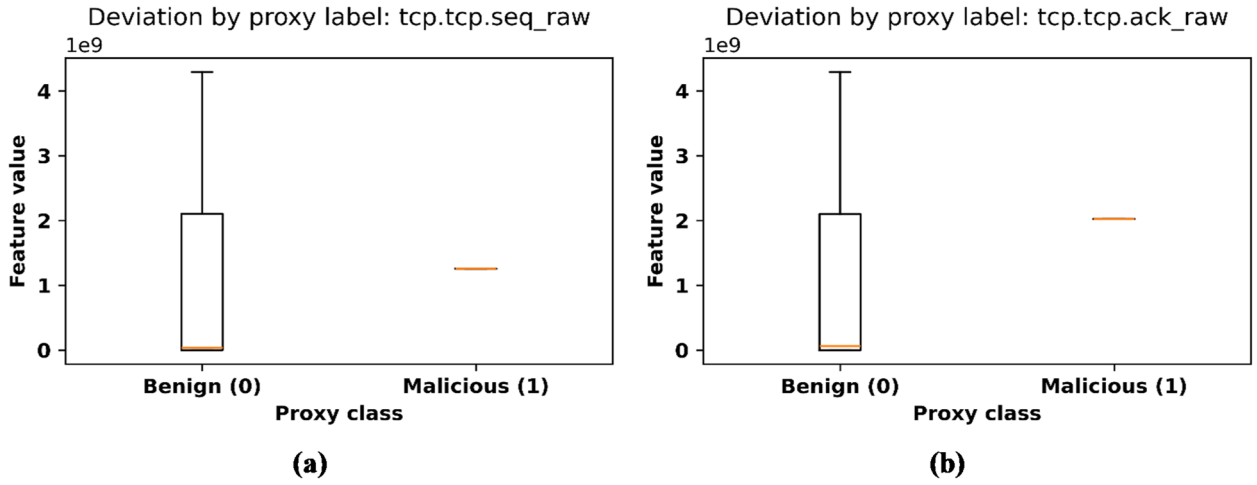


Fig. 5. Boxplots of (**a**) tcp.tcp.seq_raw and (**b**) tcp.tcp.ack_raw for proxy benign (0) vs. proxy malicious (1) samples from Isolation Forest.

Traffic aspect	Proxy benign behavior	Proxy malicious/anomalous behavior	Interpretation
TCP sequence behavior	More stable distribution	Clear deviation in proxy malicious samples	Indicates abnormal session-level behavior
TCP acknowledgment behavior	More consistent acknowledgment pattern	Different distribution in proxy malicious samples	Suggests unusual packet exchange behavior
Isolation Forest score	Mostly below the anomaly threshold	Samples identified as anomalous by the threshold	Supports proxy malicious labeling
Overall traffic pattern	Closer to normal packet behavior	Shows deviation in selected TCP-related attributes	Supports anomaly-based proxy labeling

Table 5. Traffic-level interpretation of Isolation Forest-generated proxy labels.

TONIoT Dataset

This dataset encompasses diverse data gathered from an Industrial IoT (IIoT) system. The dataset comprises telemetry data from linked devices, network traffic and operating system logs from Linux and Windows. The data was obtained from a medium-scale IoT network established by the UNSW Canberra IoT Labs and the Cyber Range. The pattern of attack was captured from a network consisting of diverse IoT devices and sensors. The assaults in the TONIoT dataset are categorised into nine distinct kinds. Each attack signifies a particular form of cyber threat[41]. The scanning assault transpires when an assailant collects information on the target system which includes the identification of open ports and services. This attack is the first step to further attacks. Cross-Site Scripting (XSS) attacks target IoT web servers by inserting harmful scripts which may compromise the authentication processes. The DoS attack inundates the system with harmful traffic which intends to render services inaccessible. The DDoS attacks threaten the target system by flooding it with an overload of connections that are made by using a network of compromised devices. One of the attacks is a backdoor attack where malware is deployed to facilitate unauthorised remote access to the infected IoT systems that support DDoS attacks using a botnet. Injection attacks occur when attackers inject malicious code into a system to execute harmful activities. These diverse types of attacks are a demonstration of the complex security risks that the TONIoT dataset will be used to detect and address.

CICIDS2018 dataset

CSE-CIC-IDS2018, also known as CICIDS2018 or CSE-CIC-IDS-2018, is a contemporary intrusion detection standard that is published by the University of New Brunswick, under the title of the Canadian Institute for Cybersecurity. It was created to overcome some of the weaknesses of older IDS datasets by creating more current benign and attack traffic in a realistic controlled setting. The traffic is recorded as packet traces which are then translated into bidirectional flow records by way of flow-meter style feature extraction. Each record is one of the network flows and has a rich collection of statistical attributes that characterize timing, packet and byte counts, header associated areas and forward and backward flow attributes[42]. This dataset consists of a large variety of attack behaviors and normal activities, which makes it appropriate to assess accuracy and operational cost when performing under high data volumes. It also presents typical IDS problems including class imbalance, redundant features, and overlapping patterns among the benign and malicious flows that promote prudent preprocessing and powerful modeling decisions.

CICIoT2023 Dataset

CICIoT2023 is a recently released large-scale IoT intrusion detection dataset that aims to simulate realistic attacks in an IoT environment. It was designed based on the IoT topology of 105 devices and malicious IoT devices were utilized to attack others. There are 33 attack types that are classified into seven general categories such as DDoS,

DoS, reconnaissance, web-based attacks, brute force, spoofing and Mirai[43]. Given that the CICIoT2023 includes a variety of diverse IoT traffic patterns, as well as modern attack scenarios, it is a suitable dataset for testing IoT intrusion detection models. To validate the general behavior of the proposed BGL-RID framework under large-scale IoT attack conditions, CICIoT23 was used in this study.

Data preprocessing

The pre-processing steps were performed in the same sequence for fair model training and evaluation. Irrelevant parameters such as timestamp, identifier and IP-like fields were removed, if applicable, as they do not offer stable predictive information for intrusion detection. To enhance the quality of data, features with a significant number of missing values were excluded, and for the remaining missing values, median imputation was employed. Label encoding was used to transform the categorical attributes into numerical values to enable the machine learning models to process them. The mapping of the categorical attributes across different datasets presented in Table 6 was used the same for the test, and if a test value for an attribute was a categorical value that didn't appear in the training data, it was mapped to an unknown category value. The dataset was then split into 80% training and 20% testing data sets, with a fixed random seed of 42. Then, the feature values were standardized using the Standard Scaler so that they have a mean of zero and a unit variance. The scaling parameters were fitted on the training set and applied to the test set. To overcome class imbalance, only SMOTE was applied to the training set. In the same way, Boruta and GFS were only fitted to the training set and the feature subset obtained from the training set was tested on the test set. This ordering was done to keep the preprocessing pipeline consistent, and to make sure that the evaluation was done on unseen test data.

Feature selection

In this section, the description of feature selection technique applied is provided. When hybrid feature selection is executed, it is usually in the form of a filter phase and then a wrapper phase or an embedded phase. The intrusion detection data has a lot of features that are correlated and redundant, thus a basic filter ranking may leave overlapping features. Wrapper methods like RFE or exhaustive subset search may also be costly in the event of large datasets and when comparing multiple classifiers[44]. Embedded methods might be sensitive to the estimator used and do not have a good cross-dataset transfer. To address these issues, we adopted a hybrid feature selection strategy that integrates Boruta and GFS techniques to identify the most informative and relevant features for IoT attack detection. The reason for employing this hybrid approach is to combine the robustness of Boruta which ensures all relevant features are captured and the efficiency of the GFS which optimizes the feature subset for model performance and computational cost.

Boruta and GFS are motivated by the fact that Boruta is oriented to all-relevant screening in order to ensure that no informative attributes are omitted, whereas GFS is oriented towards compactness by choosing the subset that offers a measurable validation improvement. Such a combination thus enhances generalization and removes redundancy in features, which directly leads to a decrease in inference time and memory footprint on IoT devices. The Boruta algorithm is a wrapper-based feature selection algorithm that is based on a random forest classifier. This is done by comparing features against their randomized analogues and determining if that feature is important. The underlying concept is to select all features, which make statistically significant effect on the predictive result. To reduce the selected feature set further, GFS was used after feature screening using Boruta. It began by empty set of features. For each iteration, one of the remaining candidate features was temporarily included in the subset of the selected features and the subset was evaluated using the Extra Trees classifier. The evaluation was carried out on the basis of 3-fold CV and accuracy score. The cross validation accuracy of the candidate feature with the best mean score was included in the selected subset. This was repeated until no further feature was discovered which gave a higher score than the one already attained. The evaluator Extra Trees was implemented using a fixed random seed of 42 to assure reproducibility. Once the final subset was determined using the training set, the same final subset of feature columns was applied to the held-out test set for the final model evaluation. Let the dataset be represented as in Eq. 1:

$$D = \{(x_i, y_i)\}_{i=1}^{N},\ x_i \in \mathbb{R}^d, y_i \in \{1,2,3,\ldots\ ..M\} \tag{1}$$

Random Forest base learner with equal weight classes and a fixed random seed is used in the implementation of Boruta so that it is reproducible. The Boruta iterations are repeated until features are placed into three states, confirmed, rejected, or tentative, as a result of repeated comparison with shadow features. The Boruta output is only retained with the confirmed features and is further fed to the second-stage selection. This technique consists of the following steps:

Dataset	Categorical feature handling
Raspberry Pi	Only numeric columns were retained, so categorical encoding was not required.
TONIoT	IP-like string columns were removed and remaining categorical features were encoded using label encoding.
CICIDS2018	Low-cardinality categorical features were handled using one-hot encoding during loading.
CICIDS2023	IP-like string columns were removed and remaining categorical features were encoded using label encoding.

Table 6. Description of Categorical feature handling across different datasets.

(i) **Shadow Feature Creation**: For each original feature, a permuted copy (shadow feature) X'_j is created as shown in Eq. 2:

$$X'_j = permute\,(X_j) \tag{2}$$

This ensures that X'_j has same distribution but no predicted relationship with the target variable y.

(ii) **Feature Important Estimation**: A Random forest classifier is trained on the extended feature set $X^* = [X, X']$. The importance of each feature is computed using Mean Decrease in Gini impurity (MDI) as represented in Eq. 3:

$$I_J = \frac{1}{T} \sum {}^{T}_{t=1} \Delta I_j^{(t)} \tag{3}$$

(iii) **Statistical Comparison**: Each original feature importance is compared to the maximum importance among its shadow counterparts I'_{max}. If $I_j >$, the feature is deemed important and if significantly less, it is rejected. Otherwise, it is tentatively kept for further iterations.

Through this process, Boruta provides a set of relevant and robust features, ensuring no potentially useful feature is discarded prematurely. To continue improving the set of features selected by Boruta and decrease the redundancy, we used the GFS technique. This method selects the subset of features which is built step-by-step, each step removing the feature that minimizes model performance. Let F denote the full feature set obtained from Boruta and $S_k \subseteq F$ be the selected subset after k iterations. The algorithm proceeds as follows:

(i) **Initialization**: Start with an empty subset $S_0 = \theta$ ..
(ii) **Feature addition Step**: At iteration $k+1$, using Eq. 4, select feature f^* that maximizes the evaluation criterion $J\,(*)$. Where $J\,(*)$ represents the model performance metric. When added to the current subset:

$$f^* = \arg \max_{f \in F \setminus S_k} J\,(S_k \cup \{f\}) \tag{4}$$

(iii) **Update Rule**: Update the subset as shown in Eq. 5:

$$S_{k+1} = S_k \cup \{f\} \tag{5}$$

Stopping error and selection threshold: Each iteration decides the candidate feature f^* by running stratified cross-validation on the training data and by computing J(.) to be the average validation score by fold. In order to prevent unimportant improvements, a minimum improvement threshold ε is used. In particular, a new feature can be accepted when:

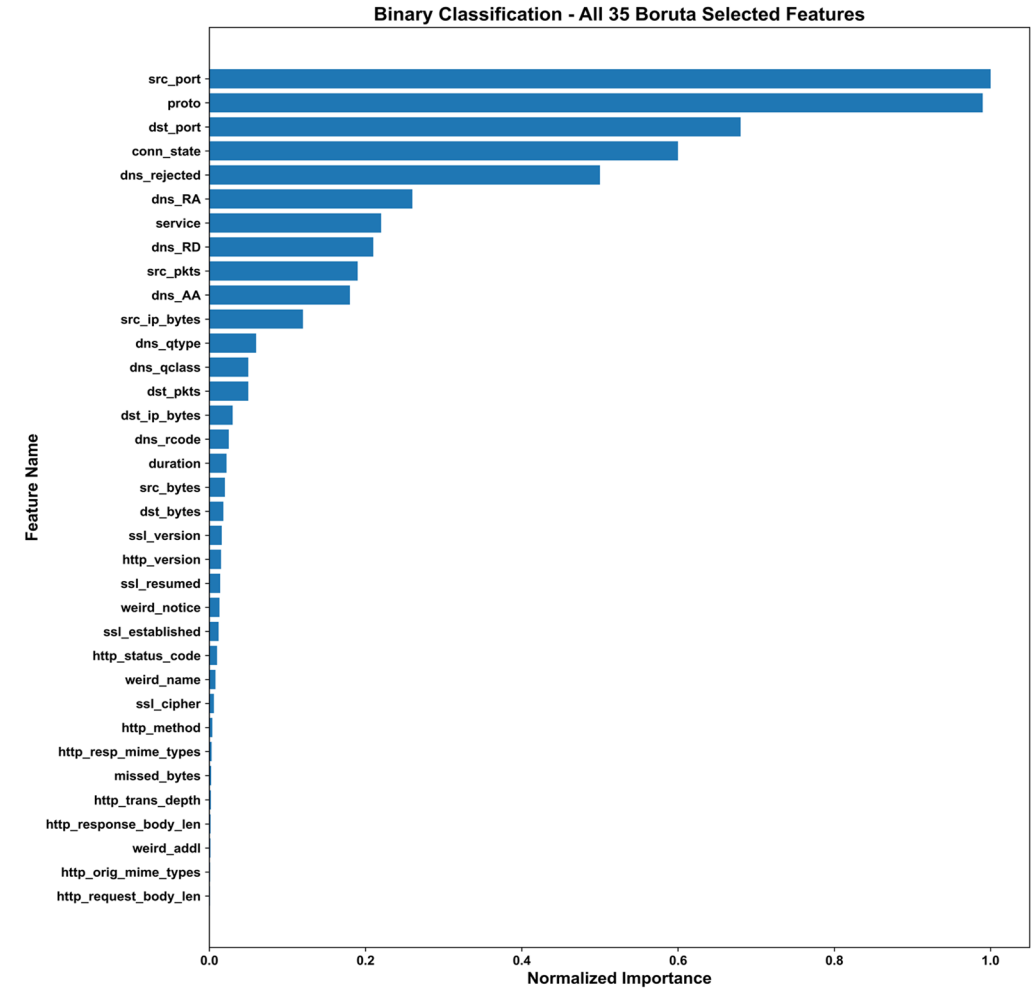
$$\max_{f \in F \setminus S_k} \left(J\,(S_k \cup \{f\}) \; - \; J\,(S_k)\,\right) \; \geq \; \epsilon \tag{6}$$

When the most suitable candidate does not improve by this minimum, the process of selection is terminated. That is why the GFS process stops when there is no remaining feature that can make some improvement with a minimum of ε or when all candidates have been used out. In the experiments, 3-fold CV = 3 is employed in GFS and ε is fixed to 0, i.e., a feature can only be accepted when it strictly improves the mean cross-validation score. ExtraTrees is the evaluator of GFS since it is fast on tabular data and gives consistent validation scores in iterative selection. For all the datasets analyzed, the final selected subsets were compact with 7 features for the Raspberry Pi dataset, 8 and 13 features for TONIoT binary and multiclass task, 11 features for CICIDS2018 and CICIoT2023. Cross-seed validation was performed in the final pipeline to check the stability and reliability of final selected feature subset. The number of features selected by each technique is summarized in Table 7 and graphically represented in Figs. 6 and 7, and 8. These final subsets represent the most discriminative features making the model both accurate and resource-efficiency which is a crucial requirement for deployment in IoT environments where memory and processing power are limited.

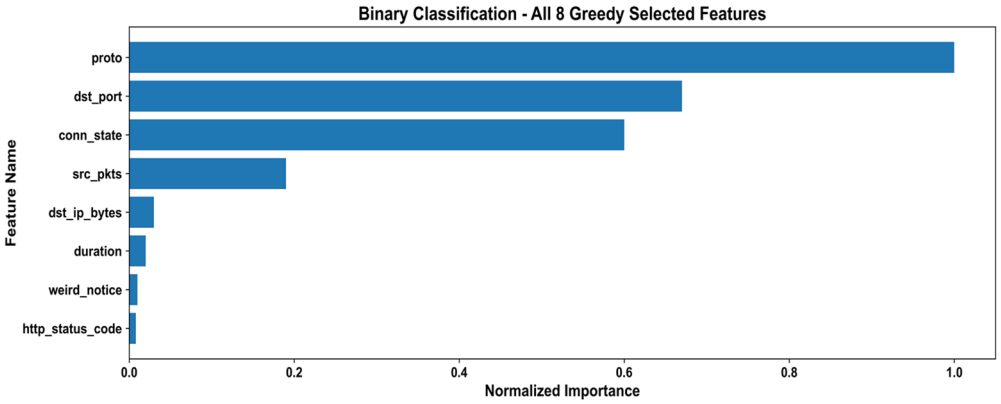
The feature selection stage is tree-based as they can capture any nonlinear relationships, feature interactions and redundant patterns but the selected feature subset was tested with both tree-based and non-tree based classifiers. This was to investigate if the selected features were discriminative in general and not just beneficial to the tree-based models. The presence of Logistic Regression, SVM and other classifiers adds to the evidence that the chosen subset is suitable for a wider model evaluation.

Feature Selection Method	Raspberry Pi	TONIoT		CICIDS2018	CICIoT2023
		Binary	Multi		
Boruta	47	35	33	70	36
Greedy Forward Selection	7	8	13	11	11

Table 7. Summary of the features selected by each dataset using different feature selection methods.



(a)



(b)

Fig. 6. (**a**) and (**b**) Features selected by the hybrid model in TONIoT Dataset in Binary Classification.

Classifiers

The classifiers detect attacks in IoT environments by learning patterns from network data and categorize the patterns into normal and malicious behaviour. These techniques work efficiently because they can process high-dimensional data and effectively learn the patterns of different sorts of attacks. These classifiers have been chosen to capture different model such as linear, kernel-based, tree and boosting ensembles, which can be fairly compared in terms of their efficiency and accuracy when using the same pipeline. The different machine learning classifiers are as follows:

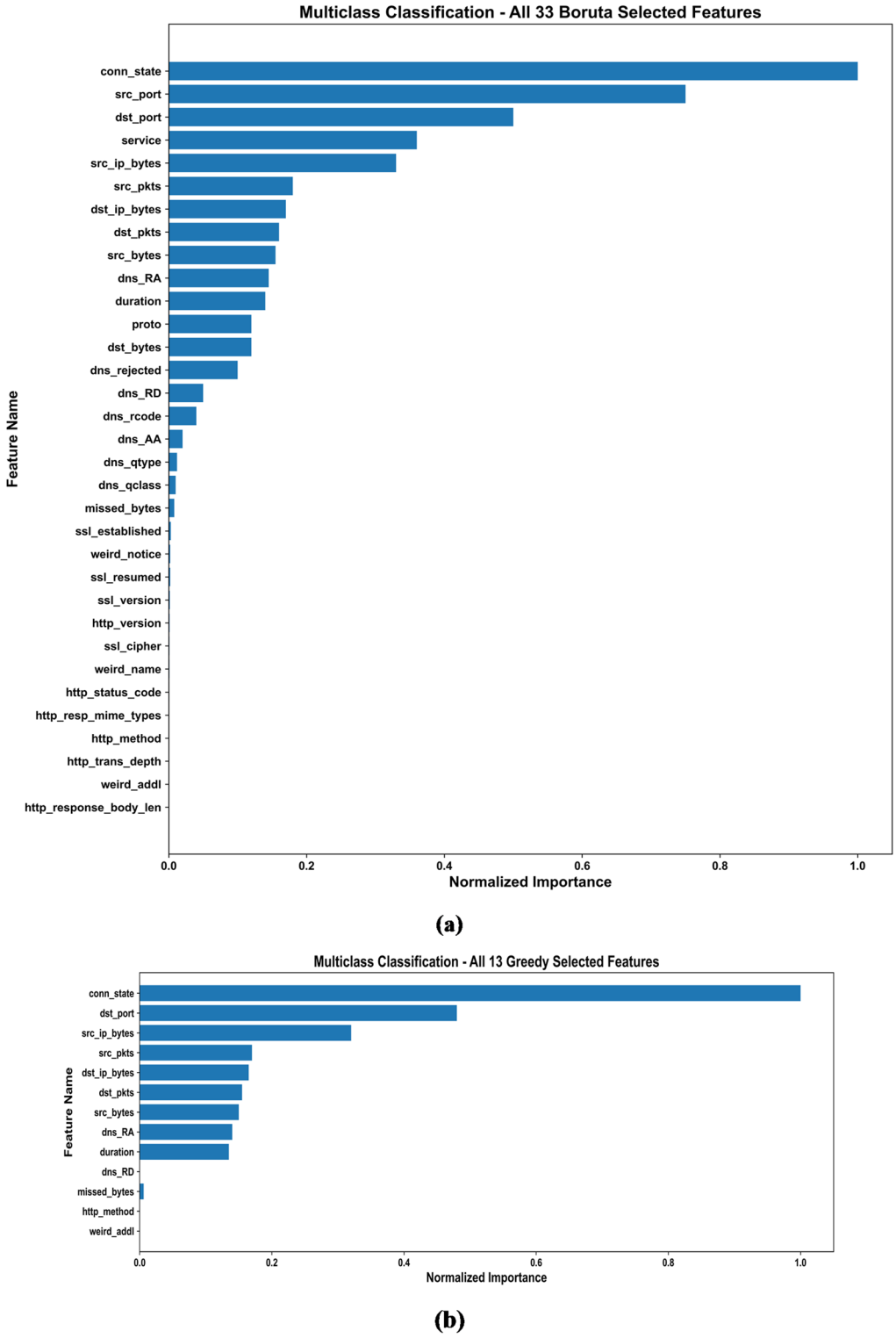


Fig. 7. (**a**) and (**b**) Features selected by hybrid model in ToNIoT Dataset in Multi-classification.

Light gradient boosting machine (LightGBM)

It is a gradient boosting framework that builds decision trees using a leaf-wise growth strategy. This technique is very effective for large-scale IoT traffic data[45]. It detects attacks by minimizing a loss function $L(y, \widehat{y})$. The model iteratively updates predictions as represented in Eq. 7:

$$\widehat{y}_{t+1} = \widehat{y}_t + \eta \cdot f_t(x) \tag{7}$$

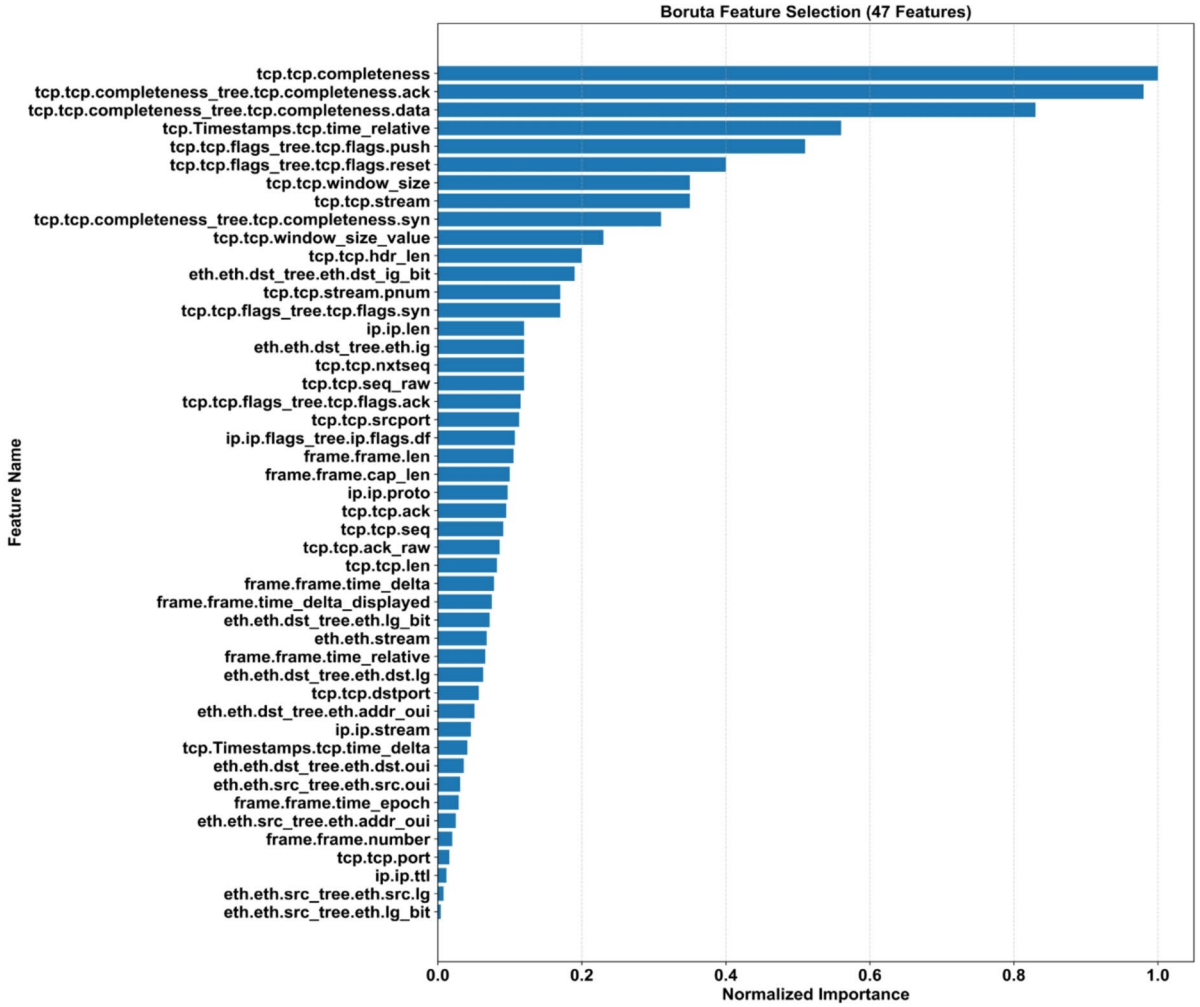


Fig. 8. Features selected by Raspberry Pi Dataset in Binary Classification.

Logistic regression

It is a linear classifier that models the probability of IoT traffic being normal or malicious using a logistic (sigmoid) function[46]. The model classifies traffic by applying a threshold on this probability and predicts the attack by using Eq. 8:

$$P\left(y=1|x\right)=\frac{1}{1+e^{-\left(w^{T}x+b\right)}} \tag{8}$$

CatBoost
It is a gradient boosting algorithm that builds decision trees while efficiently handling both categorical and numerical features in IoT traffic[47]. It minimizes the loss function. $L\left(y,\widehat{y}\right)$by adding new trees sequentially to improve prediction accuracy. The update rule is represented in Eq. 9 where $perm\left(D\right)$is a permuted dataset that ensures each prediction is trained only on earlier samples:

$$\widehat{y}_{t+1}\left(x\right) = \widehat{y}_t\left(x\right) + \eta \cdot f_t\left(x; perm\left(D\right)\right) \tag{9}$$

Random Forest
It is an ensemble learning method that builds multiple decision trees on random subsets of IoT traffic data and combines their outputs for robust attack detection[48]. This reduces overfitting and improves accuracy in identifying IoT attacks. Each tree predicts a class $h_i\left(x\right)$ and the final prediction is made by majority voting as shown in Eq. 10:

$$H\left(x\right) = mode\ \left\{h_1\left(x\right),\ h_2\left(x\right), h_3\left(x\right), \ldots\ \ldots\ h_n\left(x\right)\right\} \tag{10}$$

Gradient Boost
It builds an ensemble of weak learners (typically decision trees) sequentially, where each new tree corrects the errors of the previous ones on IoT traffic data[49]. This allows efficient detection of complex IoT attack patterns. It updates predictions by minimizing the gradient of the loss function as shown in Eq. 11:

$$\widehat{y}_{t+1}\left(x\right) = \widehat{y}_t\left(x\right) - \eta.\nabla L\left(y, \widehat{y}_t\left(x\right)\right) \tag{11}$$

Extra Trees
Extra Trees detects IoT attacks by building an ensemble of decision trees where splits are chosen randomly for features and thresholds. This method reduces variance and speed up training[50] so, final prediction is made by majority voting across all trees. At each node impurity is computed as shown in Eq. 12:

$$G\left(D\right) = 1 - \sum {}^{K}_{k=1} p_k^2 \tag{12}$$

Support vector machine
It detects IoT attacks by finding an optimal hyperplane that separates normal and malicious traffic with maximum margin[51]. The decision function is represented in Eq. 13:

$$f\left(x\right) = sign\ \left(\sum {}^{n}_{i=1} a_i y_i\ K\left(x_i, x\right) + b\right) \tag{13}$$

This enables accurate classification even in high-dimensional IoT traffic data.

Results and discussion

This section presents the experimental setup and findings of the BGL-RID model. The evaluation criteria utilized to assess the BGL-RID model in terms of detection performance and computational overhead are also described in this section.

Experimental setup

The experimental configuration for this work consists of hardware and software components which are designed to facilitate the creation of machine learning models. The device features an Intel Core i5 processor, 8GB of RAM and a 256GB SSD. This setting contributes in reducing the computation overhead occurred at the time of model training. The machine's operating system was Windows 11. Python 3.9 was the preeminent programming language while Jupyter Notebook and PyCharm were the primary environments utilised by developers. This research employed several Python libraries for assistance. We utilised Pandas and NumPy for data transformation and preparation followed by Scikit-learn for constructing machine learning models. We employed Matplotlib and Seaborn for visualization of data comprehension and to evaluate the performance of model. Evaluation is conducted on a stratified train test split with a fixed random seed. All the data-driven preprocessing algorithms such as scaling, SMOTE, Boruta, and GFS are only fitted on the training set, and applied to the test set. This inhibits data leakage and facilitates the strict reproducibility.

Evaluation criteria

We have utilized different evaluation criteria to assess the BGL-RID model such as accuracy, precision, recall, F1score, execution time, memory usage and efficiency ratio. An 80:20 random train/test split was used for all datasets, with a fixed random seed of 42 for reproducibility. The training set was employed to select features, balance the classes, and train the model, whereas the testing set was reserved to evaluate the model's performance. We have presented a novel evaluation criterion which checks the resource efficiency of a model without compromising detection accuracy. We have evaluated the model through efficiency ratio which measures accuracy, FNR, execution time and memory usage as depicted in Fig. 9 and is computed using Eq. 13. These four metrics are considered because accuracy measures overall correctness, FNR measures the critical risk of missed attacks, execution time measures CPU usage and memory usage indicates whether the model can run on limited

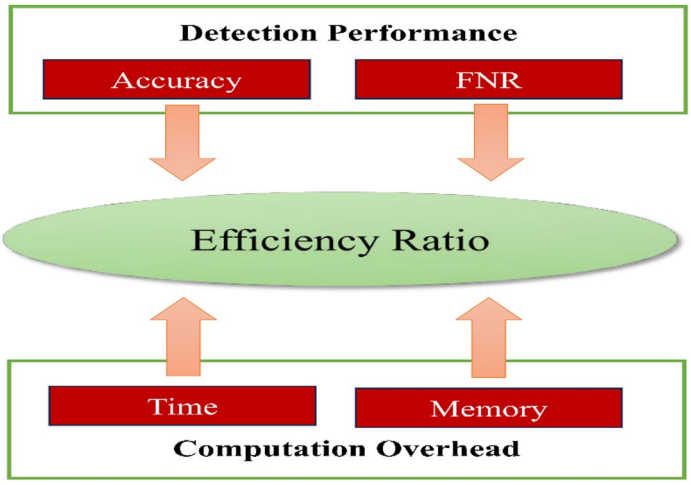


Fig. 9. Diagrammatic representation of efficiency ratio.

memory IoT devices having limited computational capacity. These parameters are integrated into a single ratio which provides a balanced assessment of the model that can work efficiently under realistic IoT constraints.

$$Efficiency\ Ratio = \frac{Accuracy\ (1 - FNR)}{Time \times\ Memory} \tag{14}$$

The quality of detection is determined using the Accuracy, Precision, Recall, FNR, and F1-score. FNR is incorporated since missed attacks are essential in IoT security. The metrics used to measure resource usage are execution time and peak memory consumed by a model. A Single Efficiency Ratio is given out to represent the trade-off between the quality of that detection and the cost of resources. Time is calculated using seconds, and Memory is calculated using peak memory in KB. An increase in Efficiency Ratio implies that it is highly detectable and incurs lower computational costs.

Experimental results

The results for binary classification on the Raspberry Pi dataset are presented in Table 8. The results are based on Isolation Forest-generated proxy labels. We can see that LightGBM model outperforms other classifiers in terms of accuracy, precision, F1-score and efficiency ratio. This demonstrates the effectiveness of the BGL-RID model, achieving an accuracy of 99.91% and an efficiency ratio of 0.0093%. This is a significant improvement over traditional models such as logistic regression and SVM which had relatively lower accuracy and higher FNR values. Although the execution time and memory usage are lowest in case of SVM and CatBoost respectively, their performance in efficiency ratio is low due to low detection accuracy. The proposed model took 1.48 s and 7240 KB of memory usage which are second lowest as compared to other models, but are highest in other metrics such as accuracy and efficiency ratio. This experimental result of BGL-RID model significantly outperforms other models in terms of computational efficiency and attack detection accuracy as well. This shows that the proposed model is highly robust for resource-constrained devices like Raspberry Pi.

The experimental results for binary classification on the TONIoT dataset displayed in Table 9 demonstrate that LightGBM achieved an accuracy of 99.72% along with FNR of 0.30% which is second highest among other models. The highest accuracy is obtained by RF and Extra Trees, but we can see the training time and memory usage are too high which leads to a low efficiency ratio which is not suitable for deployment in an IoT environment. Moreover, there is a slight difference in accuracy between RF, Extra Trees and the proposed model. With this outstanding accuracy of proposed model and highest efficiency ratio of 0.0249%, the BGL-RID model outperforms other models in terms of detection performance and computation overhead. While CatBoost and logistic regression exhibited low memory consumption and training time with memory usage of 4998 KB and training time of 0.48 s respectively, the detection accuracy and efficiency ratio are again very low in these models. On the other hand, the next lowest memory consumed and time taken is by the BGL-RID model with 5015 KB and 0.79 s respectively, which is again a very slight difference with the memory used by CatBoost and time taken by logistic regression. This experimental analysis demonstrates that the BGL-RID model outperforms other models in computational efficiency and attack detection as well. This outstanding performance of our proposed model with highest efficiency ratio proves that our proposed model shows a favorable accuracy-cost trade-off for resource-constrained IoT scenarios.

For multiclass classification, the experimental results presented in Table 10 show that the proposed model, LightGBM outperforms other models in all the evaluation metrics which shows a strong performance in detecting IoT attacks in a resource constraint IoT Environment. The LightGBM model achieved highest accuracy of 98.93% and the FNR for the proposed model was lowest as compared to other models with 1.06% which indicates effective detection across multiple classes. Moreover, in terms of training time and memory usage, the proposed model remained more efficient taking only 12.78 sec and utilizing 22,544 KB of memory which is again

Model	Accuracy	Precision	Recall	F1-Score	FNR	Execution Time (s)	Memory (KB)	Efficiency Ratio
Unbalanced Data								
Logistic Regression	99.35	95.30	36.37	52.65	63.62	1.183438	9289.995	0.00003287775
CatBoost	99.38	91.37	41.75	57.31	58.24	80.508378	2667.476	0.00000193250
Extra Trees	99.39	92.06	41.57	57.28	58.42	7.434418	15537.184	0.00000357751
Random Forest	99.39	92.46	41.75	57.53	58.24	14.811869	19976.669	0.00000140265
Gradient Boosting	99.39	92.09	41.75	57.45	58.24	11.528894	27833.075	0.00000129338
SVM	99.37	93.96	39.06	55.18	60.93	2993.236071	8282.817	0.00000001565
LightGBM	99.38	91.69	41.57	57.21	58.42	3.046124	4605.180	**0.00002945774**
Balanced Data								
Logistic Regression	94.21	98.04	90.22	93.97	9.78	1.156105	14697.844727	0.005002829
CatBoost	99.87	99.76	99.98	99.87	0.01	73.218388	5429.873047	0.0002511749
Extra Trees	99.88	99.84	99.93	99.88	0.06	19.502875	37719.154297	0.0001356985
Random Forest	99.81	99.66	99.95	99.81	0.04	36.922476	46509.235352	0.00005809861
Gradient Boosting	99.61	99.23	99.99	99.61	0.003	55.734334	53932.80957	0.00003313713
SVM	94.69	98.03	91.23	94.51	8.76	0.719096	9829.503906	0.005363441
LightGBM	99.91	99.85	99.97	99.91	0.02	1.478899	7240.342773	**0.009328671**
Balanced Train Data								
Logistic Regression	98.16	34.85	96.65	51.23	3.34	2.887222	49853.282	0.0000065915
CatBoost	99.82	85.11	99.64	91.80	0.35	74.875454	1054.012	0.0000126040
Extra Trees	99.93	95.36	97.88	96.61	2.11	8.876869	164071.522	0.00000067163
Random Forest	99.91	93.50	98.76	96.06	1.23	17.890258	249382.577	0.00000022119
Gradient Boosting	99.45	65.01	99.47	78.63	0.52	70.962261	64888.543	0.00000021485
SVM	97.66	29.92	99.82	46.04	0.17	28700.425148	67018.398	0.00000000050
LightGBM	99.86	89.20	98.94	93.82	1.05	2.995058	7239.948	**0.0000455702**

Table 8. Comparison of the BGL-RID model with the existing in Binary Classification in Raspberry Pi Dataset.

Model	Accuracy	Recall	Precision	F1-Score	FNR	Execution Time (s)	Memory (KB)	Efficiency Ratio
Unbalanced Data								
Logistic Regression	91.04	91.04	90.98	0.906321	2.3844	0.720556	6803.012	0.0001813075
CatBoost	98.79	98.79	98.81	0.988027	1.0804	92.362557	3178.837	0.000003328663
Random Forest	99.10	99.10	99.10	0.991039	0.5309	41.857201	18854.103	0.000001249123
Gradient Boosting	98.56	98.56	98.57	98.5686	1.0649	11.089167	24658.833	0.000003566226
Extra Trees	99.15	99.15	99.15	0.991539	0.5216	32.892727	15705.062	0.00000190941
SVM	94.08	94.08	94.23	93.84	0.8103	31331.277801	13259.228	0.0000000022463
LightGBM	98.79	98.79	98.81	98.79	1.1736	2.172978	3910.462	**0.000114900**
Balanced Data								
Logistic Regression	90.38	90.38	90.38	90.38	9.85	**0.48185**	8490.95312	0.0199
CatBoost	99.71	99.71	99.71	99.71	0.31	49.273451	**4998.432617**	0.000403
Random Forest	**99.80**	**99.80**	**99.80**	**99.80**	**0.25**	13.114663	27785.238281	0.000273
Gradient Boosting	99.31	99.31	99.31	99.31	0.42	19.29685	34615.93457	0.000148
Extra Trees	**99.80**	**99.80**	**99.80**	**99.80**	**0.25**	12.029908	930877.654297	0.00000889
SVM	95.627	95.62	95.85	95.62	7.86	10191.17115	18336.011719	0.000000471
LightGBM	99.72	99.72	99.72	99.72	0.30	0.7963	5015.411133	**0.0249**
Balanced Train Data								
Logistic Regression	87.62	87.62	88.20	87.84	9.82	1.132314	42716.238	0.00001633723
CatBoost	99.81	99.81	99.81	99.81	0.1708	27.476532	3490.834	0.0000103887
Random Forest	99.88	99.88	99.88	99.88	0.0931	20.008158	38797.776	0.0000012855
Gradient Boosting	99.47	99.47	99.47	99.47	0.2918	34.697596	43667.899	0.00000065461
Extra Trees	99.90	99.90	99.90	99.90	0.0745	12.733937	33765.819	0.000002321743
SVM	95.33	95.33	95.76	95.42	5.1445	18684.152398	70526.134	0.0000000006862
LightGBM	99.84	99.84	99.84	99.84	0.1304	1.233476	4946.489	**0.0001634278**

Table 9. Comparison of the BGL-RID model with state-of-the-art in binary classification in ToNIoT Dataset.

Model	Accuracy	Recall	Precision	F1-Score	FNR	Execution Time (s)	Memory (KB)	Efficiency Ratio
Unbalanced Data								
Extra Trees	97.75	97.75	97.79	97.76	4.64	16.726692	16371.244	0.00000340407
Random Forest	97.80	97.80	97.85	97.81	4.18	19.783806	19519.142	0.000002426876
CatBoost	97.75	97.75	97.83	97.77	3.77	292.135550	9607.965	0.0000003351379
Logistic Regression	51.48	51.48	57.56	46.10	56.13	38.878644	30413.173	0.0000001909739
Gradient Boosting	96.96	96.96	97.08	97.00	5.25	213.366488	66512.371	0.00000006473705
SVM	67.24	67.24	73.97	64.66	36.82	60451.185633	26075.140	0.00000000026950
LightGBM	96.10	96.10	96.19	96.12	8.05	11.641200	12482.639	**0.000006080715**
Balanced Data								
Extra Trees	98.88	98.88	98.88	98.88	1.11	43.468172	42942.084961	0.000052383
Random Forest	98.86	98.86	98.87	98.86	1.13	39.151733	50754.729492	0.000049189
CatBoost	98.81	98.81	98.82	98.81	1.18	347.246594	22760.763672	0.000012354
Logistic Regression	64.73	64.73	66.45	64.11	35.32	61.679423	68979.342773	0.000009840
Gradient Boosting	97.98	97.98	0.980104	97.9876	2.01	608.986976	126728.022461	0.0000012441
SVM	71.47	71.47	0.733883	68.6525	28.52	47749.269989	67603.697266	0.000000015825
LightGBM	98.93	98.93	0.98948	98.9375	1.06	12.780174	22544.745117	**0.0003397355**
Balanced Train Data								
Extra Trees	98.65	98.65	98.73	98.68	2.54	25.135323	47567.830	0.0000008041474
Random Forest	98.77	98.77	98.85	98.80	2.24	46.627899	55382.797	0.0000003738957
CatBoost	98.72	98.72	98.87	98.77	1.91	325.686611	11363.488	0.0000002616516
Logistic Regression	69.87	69.87	74.01	70.38	27.70	80.420639	112727.774	0.0000000557242
Gradient Boosting	98.14	98.14	98.40	98.23	2.32	663.782539	128141.274	0.0000001127045
SVM	81.71	81.71	84.38	82.42	18.27	31020.854804	90694.973	0.000000000237392
LightGBM	98.76	98.76	98.87	98.80	2.23	11.988434	15177.833	**0.000005306513**

Table 10. Comparison of the proposed model with state-of-the-art in multiclass classification in ToNIoT Dataset.

Model	Accuracy	Recall	Precision	F1-Score	FNR	Time (s)	Memory (KB)	Efficiency Ratio
Logistic Regression	94.54	94.54	94.79	94.54	1.73	1.579248	25,967.217773	0.00002265637
CatBoost	99.91	99.91	99.91	99.91	0.032	27.704661	4,489.227539	0.00000803055
Extra Trees	99.96	99.96	99.96	99.96	0.010	12.916256	14,224.732422	0.00000544048
Random Forest	99.96	99.96	99.96	99.96	0.0329	20.681985	18,509.368164	0.00000261040
Gradient Boosting	99.49	99.49	99.49	99.49	0.1745	35.92893	22,275.901367	0.00000124097
SVM	96.36	96.36	96.59	96.35	0.1127	3,861.940781	39,488.266602	0.00000000631
LightGBM	99.94	99.94	99.94	99.94	0.0327	0.865486	4,555.43457	**0.0002534185**

Table 11. Comparison of the proposed model with existing in Binary classification in CICIDS2018 Dataset.

lowest among other models. The proposed model also showed highest efficiency ratio of 0.00033%, which again demonstrates that LightGBM model is highly efficient in handling multiclass attack detection in IoT networks consisting of resource constraint devices. This outstanding performance of the LightGBM model makes it more suitable to be deployed in an IoT environment.

Robustness of the LightGBM model is further tested using CICIDS2018 dataset, the result is shown in Table 11. This dataset differs in terms of diversity in traffic, distribution of features, and attack patterns. The goal is to ensure that the LightGBM method is not dependent on dataset specific artifacts. In this dataset also LightGBM showed high Accuracy, Precision, Recall, and F1 score and low false negative rate. It also has the best efficiency profile as it achieves this performance with the lowest runtime and low memory use, which gives the highest Efficiency Ratio. These results show stable performance under both detection quality and resource constraints.

The relevance of the study to IoT was further strengthened by evaluating the proposed BGL-RID framework on CICIoT2023 which is an IoT-specific intrusion detection benchmark. That outcome is reported in Table 12, and it indicates that the suggested methodology is still efficient in case of IoT-based traffic and attack scenarios. BGL-RID has a 98.88% accuracy on CICIoT2023. Besides effective detection performance, the model had highest efficiency ratio of 1.497766e-05 that denotes a good balance between predicting quality and resource consumption. These findings prove that the framework is not only accurate in attack detection, but also resource efficient and acceptable to IoT-oriented deployment.

Model	Accuracy	Recall	Precision	F1-Score	FNR	Execution Time (s)	Memory (KB)	Efficiency Ratio
CatBoost	98.60	98.60	99.11	98.76	0.93	9.006712	11421.114	9.496494e-06
Logistic Regression	97.71	97.71	98.84	98.07	0.06	7.808398	112379.062	1.112867e-06
Extra Trees	99.02	99.02	99.13	99.06	11.88	49.651468	552545.105	3.180595e-08
Gradient Boosting	98.48	98.48	99.06	98.66	0.81	196.973822	157132.667	3.155985e-08
Random Forest	99.12	99.12	99.21	99.15	0.01	86.143147	644961.977	1.605381e-08
SVM	97.78	97.78	98.86	98.13	0.06	42005.051504	200234.785	1.161922e-10
LightGBM	98.88	98.88	99.18	98.97	3.61	3.617121	17591.368	**1.497766e-05**

Table 12. Comparison of the proposed model with state-of-the-art in Binary classification in CICIoT2023 Dataset.

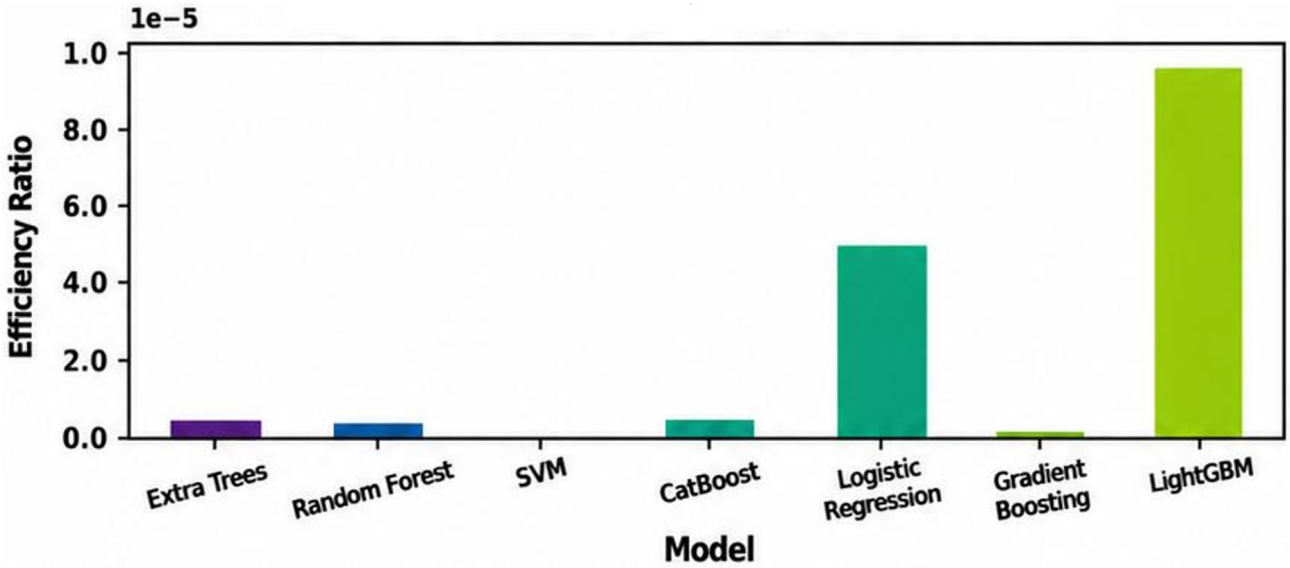


Fig. 10. Efficiency Ratio for Binary Class Classification for Raspberry Pi Dataset.

Figures 10 and 11, and 12 are the graphical representations of the performance of various models regarding the efficiency ratio with various datasets. It is evident that LightGBM is superior to other models, and the efficiency ratio is much higher, which is mainly because of the leaf-wise tree growth strategy, a good deal with high-dimensional features and optimized gradient-based learning that provides the model faster convergence and better predictive performance at lower computation time and memory usage.

Figures 13 and 14 demonstrate the ROC curve of different classifiers, and the value of the AUC of each model gave its capacity to discriminate between attack and normal classes. LightGBM model was remarkably good in both datasets with AUC of 1.000 due to its gradient-based boosting structure and non-linear interaction of features, this allows it to perform better discrimination in detecting attacks in the IoT.

As demonstrated in the confusion matrix in Figs. 15 and 16, LightGBM has a large number of correct predictions with minimal false positives and false negatives on the test set. Furthermore, the confusion matrices of the specific classes in Fig. 17 are also presented to indicate how each of the attack categories is identified in a one versus rest scenario, which justifies that misclassifications are low in the majority of classes. This result can also be supported by the PR curve in Fig. 18, as LightGBM has the highest average precision of the models compared. This behaviour suggests that LightGBM has the most reliable detection behaviour, particularly in class imbalanced settings in which the PR curve is more informative than the accuracy.

Efficiency Ratio metric is formulated to measure the quality of detection with low computational cost. In order to validate this metric, we performed pareto analysis between accuracy versus cost to find out the model which have the best tradeoff between detection and resource consumption. The Pareto trade-offs of TONIoT, CICIDS2018, Raspberry and CICIoT2023 are presented in Fig. 19. It can be seen that the proposed LightGBM is located on the Pareto frontier in all four datasets, meaning that the balance between accuracy and computational footprint is constant. Table 13 gives the resource aware findings by providing Cost_tm (Time x Memory), Acc_per_tm (Accuracy/(Time × Memory)) and the Efficiency Ratio of all models and datasets. LightGBM shows the highest trade-off as it has the highest values in Acc_per_tm and Efficiency Ratio. Table 14 lists extremely large Pearson correlation coefficients between Efficiency Ratio and Accuracy)/(Time × Memory)/ across all datasets, which reinforces the argument that the Efficiency Ratio adheres to a performance per cost baseline in addition to penalizing missed attacks (via the (1 - FNR) term). Pareto optimality is considered to be the main evidence

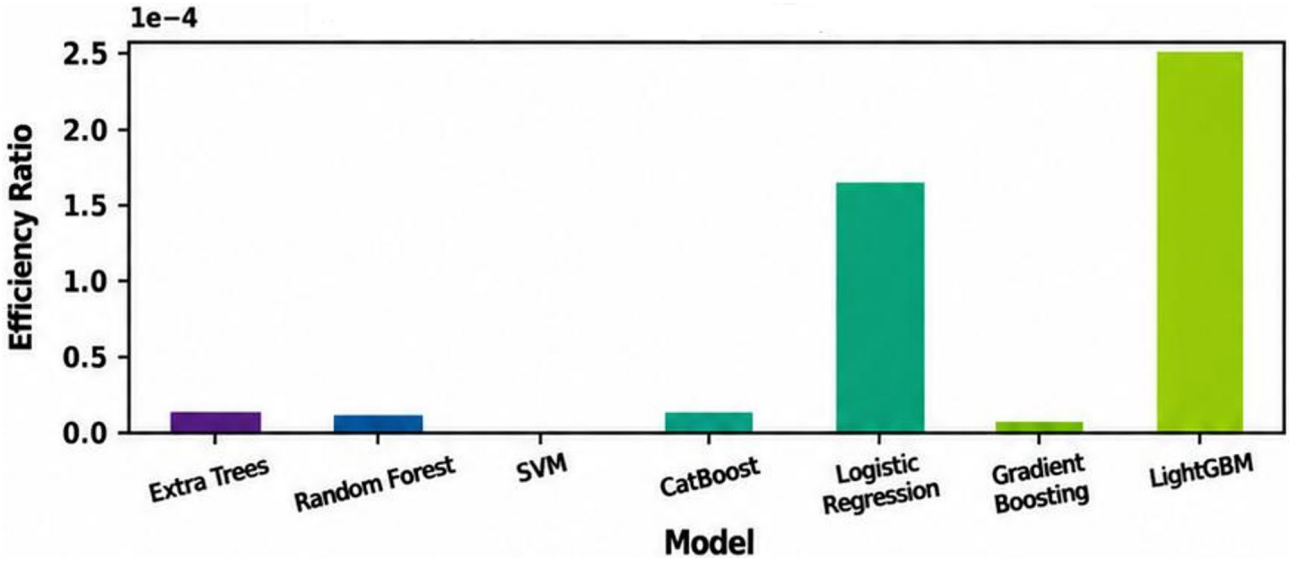


Fig. 11. Efficiency Ratio for Binary Classification in ToNIoT Dataset.

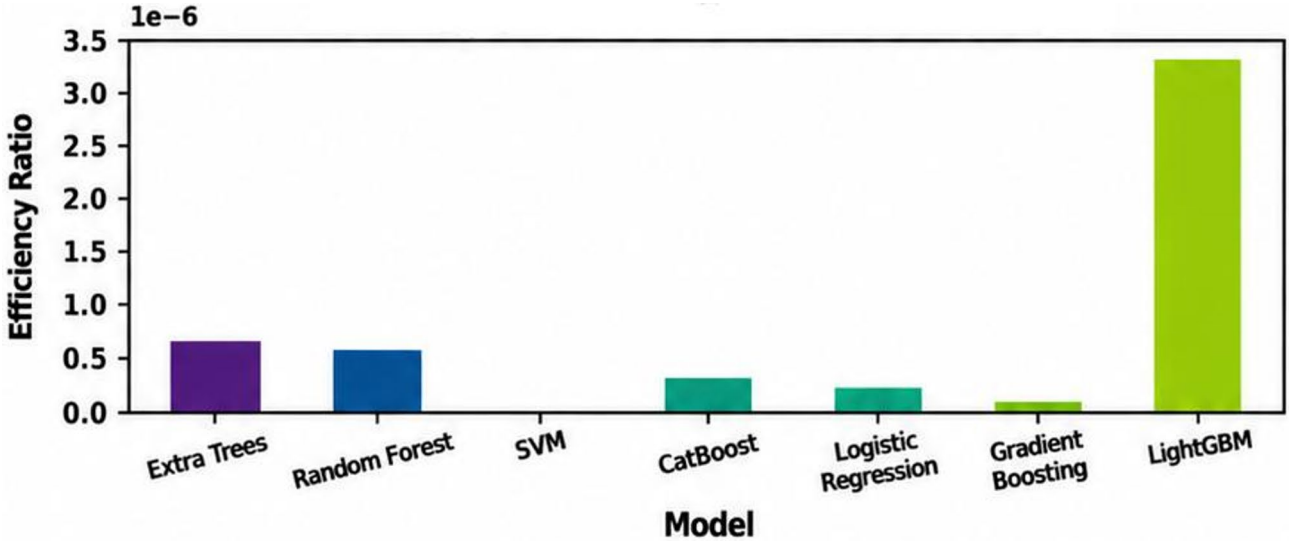


Fig. 12. Efficiency Ratio for Multiclass Classification in ToNIoT Dataset.

and the Efficiency Ratio can be validated by benchmarking it against cost_tm and Acc_per_tm across different datasets.

Cross seed validation is performed to check the robustness of the proposed model. Performance of the classifiers under three different random seeds, 21, 42 and 99. are reported in Table 15. The findings indicate that the performance of all the tree based ensemble techniques is uniformly high and stable and the confidence intervals between the reported measurements are very narrow. The proposed method outperforms other models in terms of accuracy, weighted F1, and AUC, which reflect high discrimination and high generalization Linear and margin based baselines on the other hand exhibit much weaker performance and more uncertainty that indicates increased sensitivity to the data split.

Based on the results of the ablation study as reported in Tables 16 and 17, it is concluded that the proposed hybrid configuration Boruta + GFS offers a more balanced performance in terms of detection performance and resource efficiency compared to the other configurations: without feature selection, Boruta alone, and GFS alone. Several models had high accuracy but used a greater number of features, and so were very computationally expensive without feature selection. Although Boruta was able to remove irrelevant features and still maintain high accuracy, it ended up keeping more features than it needed to, which restricted the improvement in execution time and memory usage. GFS alone yielded a smaller subset and cheaper computational cost, but also a decrease in accuracy and an increase in FNR value indicate that some relevant features may be lost too early. The hybrid Boruta + GFS pipeline preserves relevant features using Boruta and eliminates the redundant ones

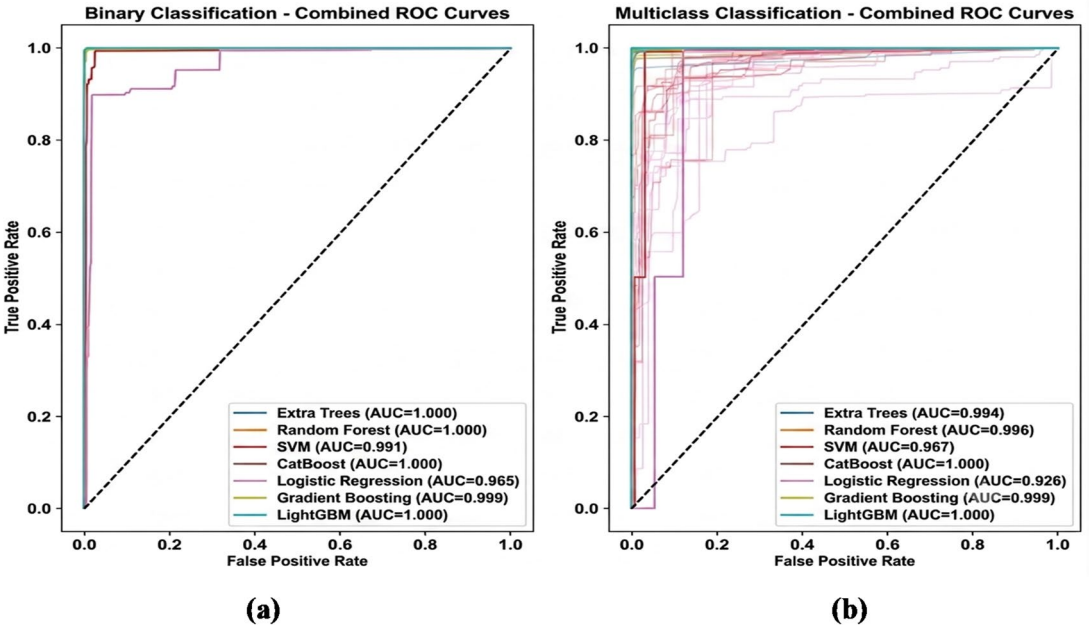


Fig. 13. ROC curve for (**a**) Binary classification (**b**) multi-classification in TONIoT Dataset.

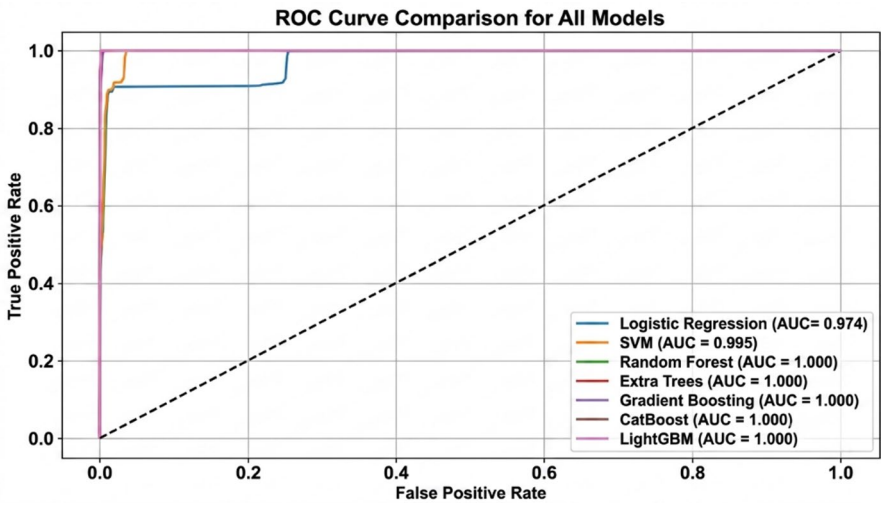


Fig. 14. ROC curve for Binary Classification in Raspberry Pi Dataset.

by the validation guided GFS. This explains why LightGBM with the hybrid feature-selection pipeline achieved 99.72% accuracy, 0.30% FNR, 0.7963 s execution time, 5015.41 KB memory usage, and the highest efficiency ratio of 0.0249. The results confirm the usefulness of the integration proposed. The methods introduced in BGL-RID, such as Boruta, GFS, SMOTE and LightGBM, are well-known techniques. But the key is in the way these methods are combined into a single resource-aware IDS workflow for the IoT environments. Boruta first conducts a feature screening using all important features, GFS generates a small feature set based on the validation process, SMOTE alleviates the class imbalance, and LightGBM is an efficient classification algorithm. The framework is structured so that it can jointly address detection accuracy, FNR, execution time, memory usage and efficiency ratio. Therefore, BGL-RID is incremental and practical while meaningful for intrusion detection in resource constrained IoT devices due to its capability to enhance the accuracy-cost trade-off.

Adversarial validation was also performed with LightGBM to investigate further dataset bias, feature leakage or train-test distribution shift as shown in Table 18. For this test, original training split samples were marked as 1, and original test split as 0. A high AUC from the adversarial classifier suggests that the train and test sets have a very distinct distribution and could be biased. An AUC near 0.50, on the other hand, means that the two partitions are hard to distinguish. The adversarial validation results of the ROC curve shown in Fig. 20 resulted in the AUC of 0.4978 for TONIoT, 0.4972 for Raspberry Pi dataset, 0.5002 for CICIDS2018 and 0.4997 for

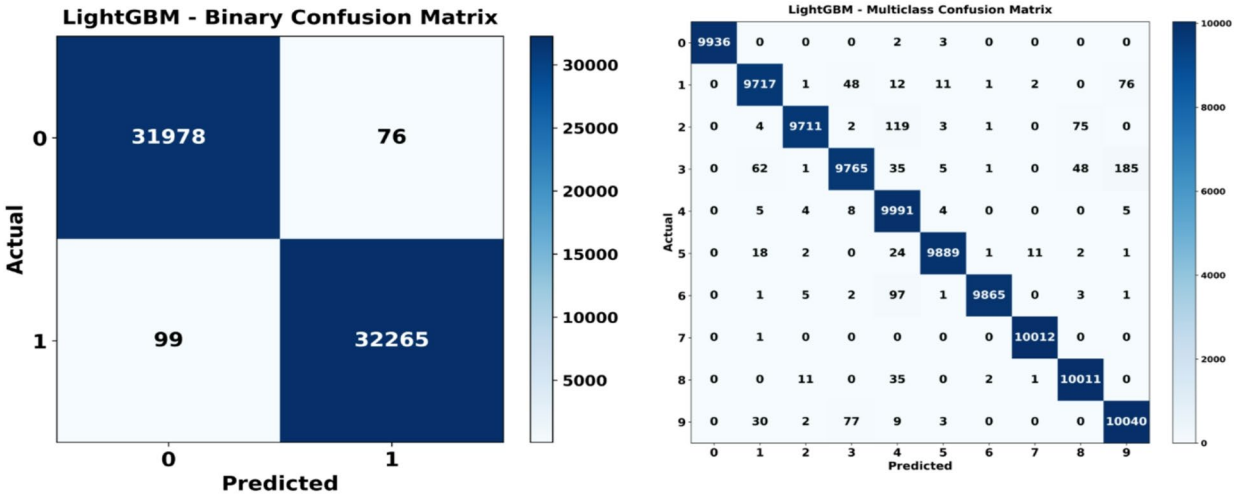


Fig. 15. Binary and multiclass classification confusion metrics of BGL-RID model of ToNIoT Dataset.

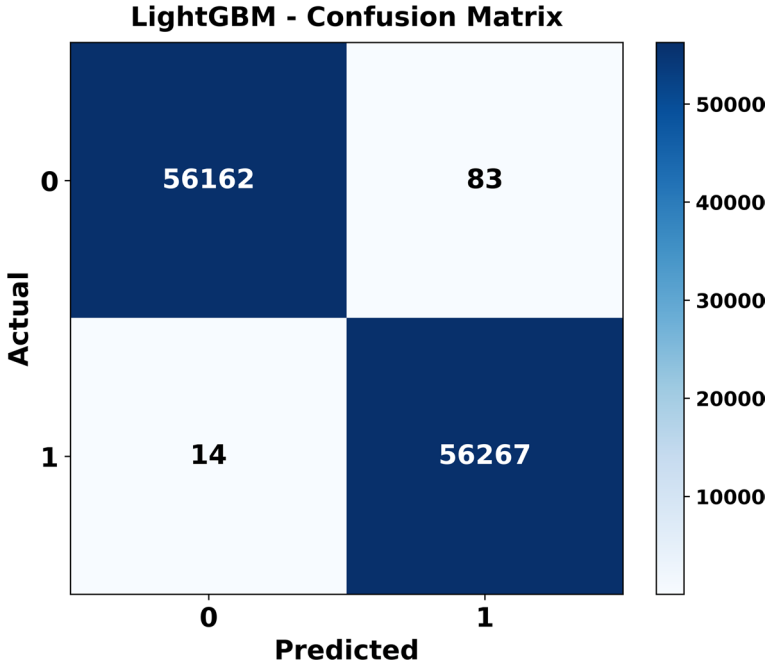


Fig. 16. Binary classification confusion metrics of BGL-RID model of Raspberry Pi dataset.

CICIoT2023. The values are very close to random separability which means that the splits of train and test sets are comparable. Hence, the high detection performance is less likely to be attributed to train-test split bias and feature leakage.

Discussion

The proposed model, BGL-RID has achieved highest efficiency ratio across different datasets which makes it highly suitable for IoT environments. It not only achieves near-optimal performance in terms of attack detection but also ensures low resource consumption which is crucial for real-world deployments where processing power and memory are limited. The inclusion of hybrid feature selection using Boruta and the GFS algorithm ensured that pertinent features were selected which have improved the performance of the model. Most hybrid feature selection based IDS methods are based on filter ranking and a generic wrapper, or they choose some fixed top k feature list without indicating how marginal each added feature is. The suggested hybrid brings in the two stage design where Boruta is used to do the stable relevance screening and GFS will be the core novelty by building the final subset using incremental utility based inclusion. Each candidate feature is introduced only when it improves the validation performance which regulates redundancy and prevents oversized feature sets. This simplifies the

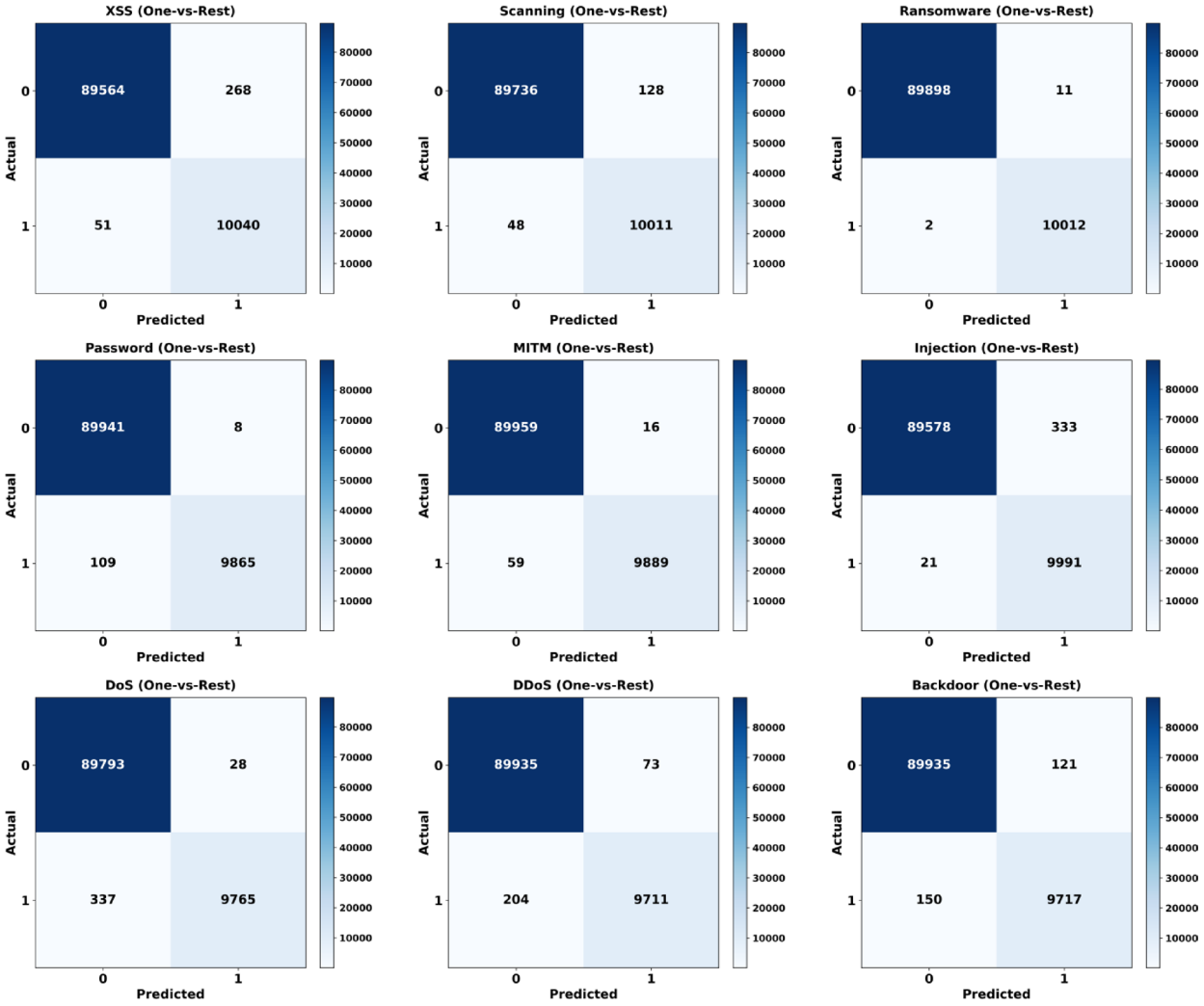


Fig. 17. Attack wise Confusion Matrics for LightGBM Classifier.

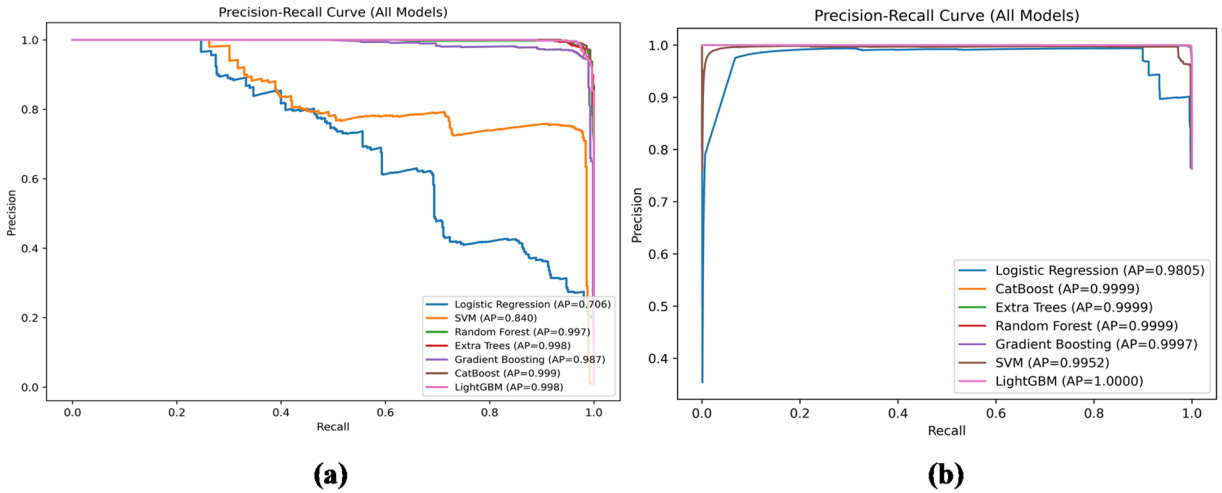


Fig. 18. PR curve for (**a**) Unbalanced Raspberry Pi Dataset (**b**) Unbalanced TONIoT Dataset.

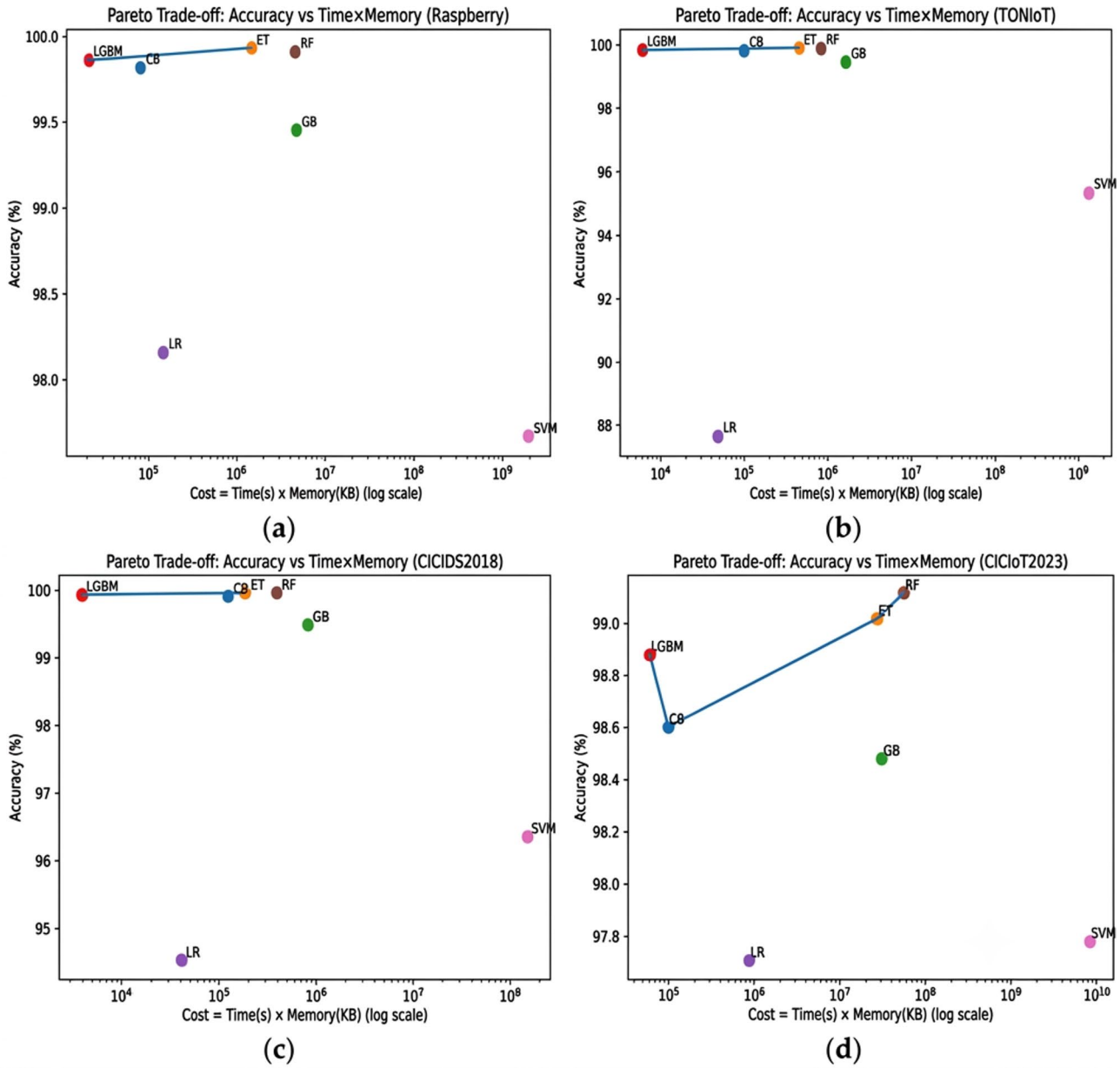


Fig. 19. Pareto trade-off on **(a)** Raspberry Pi **(b)** TONIoT, **(c)** CICIDS2018 and **(d)** CICIoT2023 Datasets.

final subset into a compact and classifier aligned form that helps in decreasing the computation cost which is significant in resource constrained IoT deployments. Additionally, the SMOTE technique effectively handled class imbalance which ensured that the model was capable of detecting rare attack patterns without suffering from overfitting. The effect of the SMOTE is also tested by running the entire pipeline without oversampling in both of the datasets. Low performance is observed in unbalanced data with the highest drop observed in minority attack categories. IoT intrusion data is highly unbalanced and infrequent attacks could be under-sampled in training. The SMOTE enhances the coverage in the training split as well as the minority-class bias and results in a better consistent detection of minority intrusions.

The components employed in BGL-RID model are structured to satisfy IoT deployment requirements. The pipeline consists of hybrid feature selection technique which does stable relevance screening at the first stage and the end subsets are created by using validation guided GFS in the second stage. Smote is used to equalize the classes to avoid over-representation of majority classes. Lastly the robustness of LightGBM classifier is tested using cross seed validation and with confidence intervals which explains that the performance observed are consistent. Hence, BGL-RID adds a reproducible and efficiency motivated IDS workflow enhancing generalisation and suitability to be deployed in IoT environment.

To ensure optimal parameter setting and comparison, the experiments were conducted in desktop CPU The proposed model was profiled during inference to record peak memory and end to end execution time. The observed peak memory was 7239.948 KB which is about 7.07 MB, and the execution time was 2.99 s. These

Dataset	Model	Cost_tm	Acc_per_tm	Efficiency Ratio
Raspberry Pi	Logistic Regression	1.439375e + 05	6.819627e-06	6.591851e-06
	CatBoost	7.891963e + 04	1.264831e-05	1.260404e-05
	Extra Trees	1.456441e + 06	6.861244e-07	6.716472e-07
	Random Forest	4.461519e + 06	2.239372e-07	2.211828e-07
	Gradient Boosting	4.604638e + 06	2.159779e-07	2.148548e-07
	SVM	1.923457e + 09	5.077318e-10	5.068686e-10
	LightGBM	2.168406e + 04	4.605225e-05	4.556870e-05
TONIoT	Logistic Regression	4.836819e + 04	1.811521e-05	1.633630e-05
	CatBoost	9.591601e + 04	1.040598e-05	1.038821e-05
	Extra Trees	4.299718e + 05	2.323408e-06	2.321677e-06
	Random Forest	7.762720e + 05	1.286662e-06	1.285464e-06
	Gradient Boosting	1.515171e + 06	6.564935e-07	6.545779e-07
	SVM	1.317721e + 09	7.234460e-10	6.862283e-10
	LightGBM	6.101375e + 03	1.636352e-04	1.634219e-04
CICIDS2018	Logistic Regression	4.100868e + 04	2.305366e-05	2.265483e-05
	CatBoost	1.243725e + 05	8.033125e-06	8.030554e-06
	Extra Trees	1.837303e + 05	5.440584e-06	5.440040e-06
	Random Forest	3.828105e + 05	2.611214e-06	2.610355e-06
	Gradient Boosting	8.003493e + 05	1.243082e-06	1.240913e-06
	SVM	1.525013e + 08	6.318633e-09	6.311512e-09
	LightGBM	3.942665e + 03	2.534834e-04	2.534005e-04
CICIoT2023	Logistic Regression	8.775004e + 05	1.113504e-06	1.112836e-06
	CatBoost	1.028667e + 05	9.585222e-06	9.496079e-06
	Extra Trees	2.743468e + 07	3.609301e-08	3.180516e-08
	Random Forest	5.555905e + 07	1.784048e-08	1.783869e-08
	Gradient Boosting	3.095102e + 07	3.181801e-08	3.156029e-08
	SVM	8.410872e + 09	1.162543e-10	1.161845e-10
	LightGBM	6.363011e + 04	1.553981e-05	1.497883e-05

Table 13. Resource-aware results across four datasets.

Dataset	Pearson correlation (Efficiency Ratio, Acc_per_tm)
Raspberry	1.0000
TONIoT	0.9999
CICIDS2018	1.0000
CICIoT2023	0.9999

Table 14. Pearson correlation between efficiency ratio and other resource aware metric.

values are within the resource envelope of Raspberry Pi 4 Model B which is available with 2 GB. Thus, the findings indicate the potential suitability of the proposed model for inference in Raspberry Pi class devices. Since the measurement of power and direct benchmarking on ARM hardware was not conducted, the edge deployment feasibility is supported in terms of runtime and memory profiling.

The advantage of BGL-RID can be better understood in contrast to tree ensembles like random forest and extra trees. These models can be extremely competitive and in certain instances slightly outperform LightGBM in terms of detection accuracy. However, the results show that this gain is obtained at a much higher memory and time cost. In case of IoT environments, these trade-offs are significant since even minor spikes in computational load can become a bottleneck to actual implementation on low-power devices. BGL-RID maintains close-to-optimal detection performance and has significantly lower runtime and memory footprint, which is the reason why its efficiency ratio is higher. The findings also suggest that linear or margin-based models like Logistic Regression and SVM are not effective since the data cannot be separated linearly and there are complex interactions between the features. Their lower performance implies IoT intrusion patterns are influenced by non-linear interactions of features that can be better explained by gradient boosting. LightGBM is especially useful in this case since the leaf-wise expansion scheme is able to capture such interactions at a relatively small computational cost. The Pareto analysis and efficiency ratio give a deployment-oriented understanding of the quality of the model. Many IoT security research works implicitly assume that models with slightly more accurate models are better, even when they are much slower or need more memory. Practically effective models are those that retain good

Model	Accuracy (95% CI)	F1-weighted (95% CI)	AUC (95% CI)
Raspberry Pi Dataset			
CatBoost	0.9985 [0.9982, 0.9987]	0.9985 [0.9982, 0.9987]	0.9998 [0.9998, 0.9999]
Extra Trees	0.9987 [0.9985, 0.9989]	0.9987 [0.9985, 0.9989]	0.9998 [0.9997, 0.9999]
Gradient Boosting	0.9971 [0.9968, 0.9974]	0.9971 [0.9968, 0.9974]	0.9996 [0.9995, 0.9997]
Logistic Regression	0.9876 [0.9860, 0.9893]	0.9876 [0.9860, 0.9893]	0.9969 [0.9967, 0.9971]
Random Forest	0.9986 [0.9984, 0.9987]	0.9986 [0.9984, 0.9987]	0.9998 [0.9997, 0.9998]
SVM	0.9850 [0.9833, 0.9866]	0.9850 [0.9833, 0.9866]	0.9919 [0.9895, 0.9943]
LightGBM	0.9987 [0.9985, 0.9989]	0.9987 [0.9985, 0.9989]	0.9999 [0.9998, 0.9999]
TONIoT Dataset			
Extra Trees	0.9975 [0.9973, 0.9977]	0.9975 [0.9973, 0.9977]	0.9998 [0.9997, 0.9999]
Random Forest	0.9975 [0.9973, 0.9977]	0.9975 [0.9973, 0.9977]	0.9998 [0.9997, 0.9999]
CatBoost	0.9971 [0.9968, 0.9974]	0.9971 [0.9968, 0.9974]	0.9998 [0.9998, 0.9999]
Gradient Boosting	0.9930 [0.9921, 0.9939]	0.9930 [0.9921, 0.9939]	0.9991 [0.9990, 0.9993]
SVM	0.7428 [0.6804, 0.8053]	0.7317 [0.6627, 0.8006]	0.7993 [0.7299, 0.8688]
Logistic Regression	0.7275 [0.6694, 0.7855]	0.7101 [0.6434, 0.7768]	0.6936 [0.6070, 0.7801]
LightGBM	0.9975 [0.9972, 0.9978]	0.9975 [0.9972, 0.9978]	0.9999 [0.9999, 0.9999]
CICIDS2018 Dataset			
Random Forest	0.9979 [0.9971, 0.9986]	0.9979 [0.9971, 0.9986]	0.9994 [0.9991, 0.9998]
Extra Trees	0.9978 [0.9970, 0.9985]	0.9978 [0.9970, 0.9985]	0.9992 [0.9987, 0.9997]
XGBoost	0.9975 [0.9968, 0.9982]	0.9975 [0.9968, 0.9982]	0.9997 [0.9994, 0.9999]
CatBoost	0.9975 [0.9968, 0.9982]	0.9975 [0.9968, 0.9982]	0.9996 [0.9994, 0.9998]
Gradient Boosting	0.9926 [0.9920, 0.9932]	0.9926 [0.9920, 0.9932]	0.9987 [0.9985, 0.9990]
SVM	0.9413 [0.9382, 0.9444]	0.9412 [0.9381, 0.9443]	0.9810 [0.9776, 0.9844]
LightGBM	0.9979 [0.9972, 0.9986]	0.9979 [0.9972, 0.9986]	0.9996 [0.9994, 0.9999]
CICIoT2023 Dataset			
Random Forest	0.9853 [0.9847, 0.9858]	0.9868 [0.9864, 0.9872]	0.9967 [0.9965, 0.9968]
Gradient Boosting	0.9804 [0.9795, 0.9812]	0.9832 [0.9825, 0.9838]	0.9944 [0.9941, 0.9948]
CatBoost	0.9800 [0.9791, 0.9809]	0.9829 [0.9822, 0.9836]	0.9958 [0.9954, 0.9961]
Extra Trees	0.9790 [0.9780, 0.9800]	0.9822 [0.9814, 0.9829]	0.9961 [0.9957, 0.9964]
SVM	0.9751 [0.9743, 0.9759]	0.9792 [0.9787, 0.9798]	0.9932 [0.9926, 0.9937]
Logistic Regression	0.9746 [0.9738, 0.9754]	0.9789 [0.9783, 0.9794]	0.9942 [0.9938, 0.9945]
LightGBM	0.9842 [0.9836, 0.9848]	0.9860 [0.9855, 0.9864]	0.9956 [0.9947, 0.9965]

Table 15. Classifier performance reported with 95% Confidence Intervals (CI).

performance in detection with realistic computational constraints. The Pareto-front behavior of LightGBM on different datasets thus contributes to the point that BGL-RID is not only accurate, but operationally efficient.

A literature-based comparison of the proposed work with existing studies in the field of IoT attack detection is presented in Table 19, and the reported results are aligned with the corresponding datasets. In study[52], the Symmetry Hybrid NNIDS is based on staged inference, offering LightGBM as traffic level and MobileNetV2 as packet level, to trade-off accuracy and cost on an IoT dataset. The suggested BGL-RID does not necessitate a deep packet level model. It maintains the pipeline lightweight and concentrates on small feature sets and classical learners that can be executed on the edges. In study[53] FSLLM framework targets feature redundancy and class imbalance using a multi stage feature selection pipeline, then fine tunes LLMs to generate additional attack samples, and finally applies an FL improved LightGBM classifier across multiple IoT datasets. Conversely, the study presents an IDS that is structured to be resource efficient for edge devices and evaluates various baseline models using accuracy and efficiency ratio metric to quantify the tradeoff between detection quality and resource cost which is essential for edge devices. Recent deep IDS IoMT research has an excellent detection performance but is based on heavier deep architecture. The XAI-motivated multi attention DeepCRNN in[54]and the advanced deep learning IDS frameworks in[55] are highly accurate on IoMT data, but they are not designed to run on low resources edges. Conversely, the proposed BGL-RID does not rely upon deep packet level models but rather has compact selected features and lightweight learners that can be deployed on resource limited devices.

In[56], a hybrid feature selection framework on ToN-IoT eliminated IP and port bias and generated a small five-feature set in lightweight intrusion detection. The MI2G-based IDS[57] applied the information-theoretic feature selection to minimize training cost and still maintain high detection performance with classical models. An AI-based software-defined IoT framework with a combination of SHAP, recursive feature removal, GNNs, Transformers, PPO, and privacy-preserving federated learning was used to adaptively detect and mitigate intrusion[58]. These studies were reported to have strong results, but some of them used more complicated optimization or deep architectures. Conversely, the proposed BGL-RID is lightweight and more appropriate in terms of edge deployment as it explicitly assesses the trade-off between detection rate and computing power.

Model	Accuracy	Precision	Recall	F1-Score	FNR	Execution Time (s)	Memory (KB)	Efficiency Ratio
No Feature Selection								
Extra Trees	99.93	99.93	99.92	99.93	0.0776	20.3322	144205.17	3.400000e-07
Random Forest	99.92	99.9	99.94	99.92	0.0621	15.7183	182111.17	3.500000e-07
CatBoost	99.85	99.9	99.8	99.85	0.1956	31.7622	3167.84	9.900000e-06
Gradient Boosting	99.6	99.56	99.63	99.6	0.3664	66.5042	63796.58	2.300000e-07
SVM	97.95	98.64	97.25	97.94	2.7508	13729.7283	76501.67	0.000000e + 00
Logistic Regression	93.86	91.43	96.79	94.03	3.2072	1.8529	8354.81	5.869000e-05
LightGBM	99.89	99.87	99.9	99.89	0.1025	1.951	4804.66	**1.064500e-04**
Boruta Feature Selection								
Random Forest	99.93	99.92	99.93	99.93	0.0652	18.6791	105852.08	5.100000e-07
Extra Trees	99.93	99.94	99.93	99.93	0.0714	13.9598	105688.27	6.800000e-07
CatBoost	99.84	99.87	99.8	99.84	0.2018	26.6774	1280.17	2.917000e-05
Gradient Boosting	99.6	99.56	99.63	99.6	0.3664	43.5332	105688.43	2.200000e-07
SVM	97.98	98.64	97.3	97.96	2.7011	11225.7078	142995.58	0.000000e + 00
Logistic Regression	93.87	91.45	96.79	94.05	3.2072	2.004	140916.94	3.220000e-06
LightGBM	99.89	99.88	99.9	99.89	0.1025	1.6193	74689.62	**8.250000e-06**
Greedy Feature selection								
Extra Trees	99.78	99.89	99.67	99.78	0.3322	9.102	109829.67	9.900000e-07
Random Forest	99.77	99.87	99.67	99.77	0.326	9.0828	165247.2	6.600000e-07
CatBoost	99.68	99.79	99.56	99.68	0.4409	49.0041	3167.96	6.390000e-06
Gradient Boosting	99.25	99.06	99.43	99.25	0.5651	22.14	34597.73	1.290000e-06
SVM	95.75	98.81	92.61	95.61	7.3861	10594.9889	18187.08	0.000000e + 00
Logistic Regression	90.62	90.58	90.68	90.63	9.3235	0.3815	8343.56	2.581500e-04
LightGBM	99.69	99.8	99.57	99.69	0.4253	0.7687	4750.83	**2.718200e-04**

Table 16. Ablation study for binary classification on TONIoT Dataset.

Model	Accuracy	Precision	Recall	F1-Score	FNR	Execution Time (s)	Memory (KB)	Efficiency Ratio
No feature Selection								
Extra Trees	99.23	99.23	99.23	99.23	0.771	42.3803	219712.58	1.100000e-07
Random Forest	99.17	99.17	99.17	99.17	0.834	29.9016	281823.35	1.200000e-07
CatBoost	98.93	98.94	98.93	98.93	1.071	412.3018	4835.07	4.900000e-07
Gradient Boosting	98.3	98.32	98.3	98.3	1.701	1273.5704	164081.68	0.000000e + 00
SVM	82.13	82.47	82.13	81.84	17.873	42398.2586	118756.37	0.000000e + 00
Logistic Regression	79.07	80.01	79.07	78.94	20.934	91.9423	69045.3	1.000000e-07
LightGBM	99.11	99.12	99.11	99.11	0.886	13.6078	15190.34	**4.750000e-06**
Boruta Feature Selection								
Extra Trees	99.21	99.22	99.21	99.21	0.79	30.3502	205409.26	1.600000e-07
Random Forest	99.14	99.15	99.14	99.14	0.86	28.528	240029.76	1.400000e-07
CatBoost	98.91	98.92	98.91	98.91	1.088	487.1586	4835.82	4.200000e-07
Gradient Boosting	98.18	98.19	98.18	98.18	1.824	988.1836	135956.25	1.000000e-08
SVM	83.72	84.07	83.72	83.47	16.281	34695.7206	62570.12	0.000000e + 00
Logistic Regression	78.96	79.91	78.96	78.83	21.04	100.9499	68996.12	9.000000e-08
LightGBM	99.09	99.1	99.09	99.09	0.911	15.5941	15219.65	**4.140000e-06**
Greedy Feature Selection								
CatBoost	98.77	98.78	98.77	98.77	1.235	337.0559	4835.01	6.000000e-07
Extra Trees	98.76	98.77	98.76	98.76	1.237	33.2908	184940.05	1.600000e-07
Random Forest	98.75	98.76	98.75	98.75	1.248	22.8186	253303.38	1.700000e-07
Gradient Boosting	98.01	98.03	98.01	98.01	1.989	688.194	128143.16	1.000000e-08
SVM	73.32	77.45	73.32	72.62	26.679	44035.8486	51220.93	0.000000e + 00
Logistic Regression	65.70	67.70	65.70	65.04	34.301	68.8722	68978.29	9.000000e-08
LightGBM	98.89	98.90	98.89	98.89	1.107	12.2367	15225.45	**5.250000e-06**

Table 17. Ablation study for multiclass classification on TONIoT Dataset.

Dataset	LightGBM adversarial AUC
TONIoT	0.4978
Raspberry	0.4972
CICIDS2018	0.5002
CICIoT2023	0.4997

Table 18. Adversarial Validation Results.

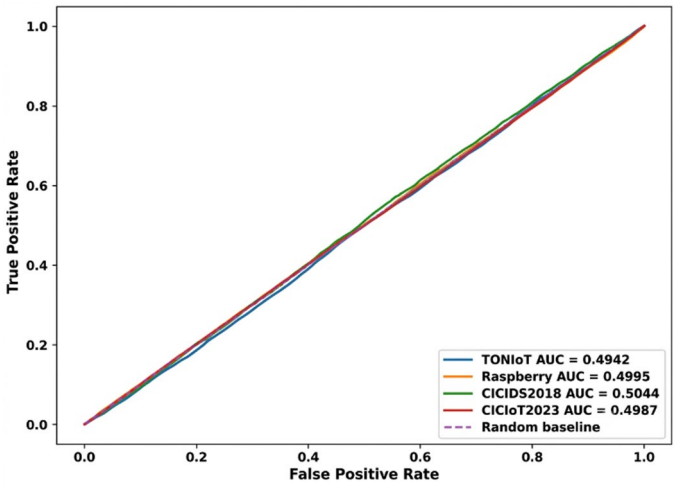


Fig. 20. LightGBM adversarial ROC curves across different datasets.

Ref.	Dataset	Methods	Year	Binary Classification				Multiclass classification			
				Acc%	Prec%	Rec%	F1-Sc%	Acc%	Prec%	Rec%	F1-Sc%
[52]	ACI-IoT-2023	Hybrid NNIDS	2025	-	-	-	-	94.00	90.00	92.00	90.00
[53]	NF-ToN-IoT-v2	FSLLM	2025	99.00	-	-	99.00	95.60	-	-	95.60
[54]	WUSTL-HDRL-2024	DeepCRNN	2025	99.79	-	-	99.66	98.23	-	-	98.12
[55]	ECU-IoHT	Advanced deep learning IDS	2025	99.90	-	-	-	-	-	-	-
[56]	TONIoT	Hybrid Feature Selection and Decision Tree	2026	98.60	98.35	98.65	98.50	97.20	92.04	91.83	91.89
[57]	CICIDS2018	MI2G	2022	99.58	99.58	99.58	99.58	-	-	-	-
[58]	CICIoT2023	GNN + Transformer + PPO + ADP + GJO	2025	98.70	-	-	98.60	-	-	-	-
	TONIoT			95.80	-	-	95.80	-	-	-	-
Proposed Model	**TONIoT**	BGL-RID	-	**99.72**	**99.72**	**99.72**	**99.72**	**98.94**	**98.94**	**98.95**	**98.94**
	CICIDS2018			**99.94**	**99.94**	**99.94**	**99.94**	-	-	-	-
	CICIoT2023			**98.88**	**98.88**	**99.18**	**98.97**	-	-	-	-
	Raspberry Pi			**99.91**	**99.85**	**99.97**	**99.91**	-	-	-	-

Table 19. Comparison with the state-of-the-art work and BGL-RID model.

Limitation and future scope

The limitations of the proposed model are as follows, which can be incorporated in future work.

- We have evaluated the proposed model on four datasets, more datasets with diverse attack patterns can be utilized to improve the generalizability of the proposed model.
- SMOTE was employed in balancing the classes. Even though it is effective, it can produce less realistic samples and can also be weak with class overlap. Future research can focus on hybrid approaches and GAN-based balancing.
- All experiments were executed on a desktop machine and the only evidence of deployment is the runtime and maximum memory. These findings indicate potential suitability for edge inference though they do not verify in-board latency, power consumption and thermal stability on ARM boards.

- The integration of the proposed model into a multi-layered security framework that combines an attack detection system with other security mechanisms such as intrusion prevention systems (IPS) and firewalls. This will ensure a more comprehensive defense strategy in IoT security.
- Interpretability of the proposed model using XAI which could further improve its adoption in critical sectors like healthcare or industrial IoT, where transparency and trust in model decisions are important.

Conclusion

In the evolving technology of IoT devices, the limited resources of these devices have been a major concern. This paper introduces a BGL-RID model for IoT attack detection which is specifically optimized for resource-constrained environments. The inclusion of hybrid feature selection using Boruta and GFS algorithms highly contributed to improving attack detection accuracy and optimizing the resource consumption. The results show the proposed model is consistent on the evaluated datasets, which demonstrates the potential application in resource constrained IoT environments. The proposed model is evaluated in terms of attack detection performance and resource consumption as well. For binary classification, the model achieved an accuracy of 99.91% from the proxy-labeled Raspberry Pi dataset. Whereas, in multiclass classification, it achieved 98.93% accuracy from the TONIoT dataset. In addition, the model achieved 99.94% accuracy on the CICIDS2018 dataset and 98.88% accuracy on the IoT-specific CICIoT2023 dataset. The execution time and memory consumption of the proposed model were also optimized which showed minimal memory requirement and faster execution time. These findings indicate the model's potential suitability for resource-aware IoT intrusion detection. Despite the impressive performance of other models, the proposed BGL-RID model outperformed them in terms of efficiency ratio that equally measures detection performance and computational efficiency which makes it a promising solution for resource-aware IoT intrusion detection. These results validate the robustness of the proposed approach which shows its potential to secure IoT environments effectively while considering the limitations of the IoT devices. Future work will involve evaluation of BGL-RID in real-time IoT systems with live network traffic and a variety of attack types. It will also investigate adaptive feature selection, federated learning, deployment to small edge devices, and interpretable AI, which will enhance the reliability and understanding of attack detection.

Data availability

The datasets analyzed during the current study are publicly available in the TONIoT Repository: https://research.unsw.edu.au/projects/toniot-datasets Raspberry Pi dataset is a generated dataset which can be made available on reasonable request. CICIDS2018 dataset: https://www.unb.ca/cic/datasets/ids-2018.html CICIoT2023 dataset: https://www.unb.ca/cic/datasets/iotdataset-2023.html.

Received: 1 January 2026; Accepted: 18 June 2026
Published online: 05 August 2026

References

1. Lai, T., Farid, F., Bello, A. & Sabrina, F. Ensemble learning based anomaly detection for IoT cybersecurity via Bayesian hyperparameters sensitivity analysis. *Cybersecurity* **7** (1), 1–18. https://doi.org/10.1186/s42400-024-00238-4 (2024).
2. Fu, X., Zhou, N., Jiao, L., Li, H. & Zhang, J. The robust deep learning–based schemes for intrusion detection in Internet of Things environments. *Ann. des. Telecommun Telecommun*. **76**, 5–6. https://doi.org/10.1007/s12243-021-00854-y (2021).
3. Ji, R., Padha, D., Singh, Y. & Sharma, S. Review of intrusion detection system in cyber-physical system based networks: Characteristics, industrial protocols, attacks, data sets and challenges. *Trans. Emerg. Telecommun Technol.* **35** (9), e5029. https://doi.org/10.1002/ett.5029 (Sep. 2024).
4. Sumathi, D. G. & Green, A. I. S. and Sustainable Innovations and Future Directions, *Adv. Comput. Sci. Bridg. AI, Netw. Emerg. Technol.*, no. Ml, pp. 1–13, (2024).
5. Safwan, H. et al. An IoT Environment Based Framework for Intelligent Intrusion Detection. *Comput. Mater. Contin.* **75** (2), 2365–2381. https://doi.org/10.32604/cmc.2023.033896 (2023).
6. Selva Kumar, V. et al. Adaptive distributed honeypot detection network for enhanced cybersecurity against DoS and DDoS attacks. *Results Eng.* **26** https://doi.org/10.1016/j.rineng.2025.105521 (2025).
7. Alam, M., Lone, A. N. & Mustajab, S. Empowering IoT security: Enhancing applications through edge computing and blockchain. *Blockchain IoT Syst. Concept. Framew. Appl.* 166–179. https://doi.org/10.1201/9781003460367-11 (2024).
8. Muthanna, M. S. A., Alkanhel, R., Muthanna, A., Rafiq, A. & Abdullah, W. A. M. Towards SDN-Enabled, Intelligent Intrusion Detection System for Internet of Things (IoT). *IEEE Access.* **10**, 22756–22768. https://doi.org/10.1109/ACCESS.2022.3153716 (2022).
9. Choudhary, V. & Tanwar, S. Generation & evaluation of datasets for anomaly-based intrusion detection systems in IoT environments. *Multimed Tools Appl.* **83**, 84331–84355. https://doi.org/10.1007/s11042-024-19066-2 (2024).
10. Khan, M. Z., Bokhari, M. U. & Khan, T. F. Internet of Things Security: An Experimental Analysis of Detecting Denial of Service Attacks with Machine Learning, in *1st International Conference on Innovative Engineering Sciences and Technological Research, ICIESTR 2024 - Proceedings*, pp. 1–6. https://doi.org/10.1109/ICIESTR60916.2024.10798334 (2024).
11. Ullah, F. et al. Privacy-Aware Secure Data Auditing for Cloud-Based Intelligence of Things Environment. *IEEE Internet Things J.* **12** (11), 15288–15303. https://doi.org/10.1109/JIOT.2025.3528117 (2025).
12. Dhinakaran, D., Srinivasan, L., Udhaya Sankar, S. M. & Selvaraj, D. Quantum-Based Privacy-Preserving Techniques for Secure and Trustworthy Internet of Medical Things: an Extensive Analysis. *Quantum Inf. Comput.* **24**, 3–4. https://doi.org/10.26421/QIC24.3-4-3 (2024).
13. Khan, A. A. et al. Privacy Preserved and Decentralized Smartphone Recommendation System. *IEEE Trans. Consum. Electron.* **70** (1), 4617–4624. https://doi.org/10.1109/TCE.2023.3323406 (2024).
14. Dhinakaran, D., Selvaraj, D., Dharini, N., Edwin Raja, S. & Priya, C. S. L. Computing with Quantum Key Distribution. *Int. J. Intell. Syst. Appl. Eng.* **12** (2), 286–300 (2024). Towards a Novel Privacy-Preserving Distributed Multiparty Data Outsourcing Scheme for Cloud.
15. Ramana, T. V., Thirunavukkarasan, M., Mohammed, A. S., Devarajan, G. G. & Nagarajan, S. M. Ambient intelligence approach: Internet of Things based decision performance analysis for intrusion detection. *Comput. Commun.* **195**, 315–322. https://doi.org/10.1016/j.comcom.2022.09.007 (2022).

16. Sumathi, S. & Rajesh, R. Comparative Study on TCP SYN Flood DDoS Attack Detection: A Machine Learning Algorithm Based Approach. *WSEAS Trans. Syst. Control*. **16**, 584–591. https://doi.org/10.37394/23203.2021.16.54 (2021).
17. Nagarajan, S. M., Deverajan, G. G., Bashir, A. K., Mahapatra, R. P. & Al-Numay, M. S. IADF-CPS: Intelligent Anomaly Detection Framework towards Cyber Physical Systems. *Comput. Commun.* **188**, 81–89. https://doi.org/10.1016/j.comcom.2022.02.022 (2022).
18. Ozaif, M., Mustajab, S. & Alam, M. Navigating Challenges in IoT: Applications, Limitations, Tools and Open Research Direction, *Proc. – 2024 13th IEEE Int. Conf. Commun. Syst. Netw. Technol. CSNT 2024*, pp. 525–531, https://doi.org/10.1109/CSNT60213.2024.10546141 (2024).
19. Tama, B. A., Comuzzi, M. & Rhee, K. H. A Two-Stage Classifier Ensemble for Intelligent Anomaly-Based Intrusion Detection System. *IEEE Access*. **7**, 94497–94507. https://doi.org/10.1109/ACCESS.2019.2928048 (2019).
20. Ji, R., Kumar, N. & Padha, D. CNN-GWO-voting & hybrid: ensemble learning inspired intrusion detection approaches for cyber-physical systems. *Proc. Indian Natl. Sci. Acad.* **91**(3), 848–862 https://doi.org/10.1007/s43538-024-00372-0 (2025).
21. Nazir, A., Memon, Z., Sadiq, T., Rahman, H. & Khan, I. U. A Novel Feature-Selection Algorithm in IoT Networks for Intrusion Detection. *Sensors* **23**, 1–19. https://doi.org/10.3390/s23198153 (2023).
22. Saied, M., Adjogble, F., Guirguis, S., Hemmji, M. & Warschat, J. A Framework for Systematic Scientific Research Management, in *PICMET 2023 - Portland International Conference on Management of Engineering and Technology: Managing Technology, Engineering and Manufacturing for a Sustainable World, Proceedings*, pp. 1–16. https://doi.org/10.23919/PICMET59654.2023.10216819 (2023).
23. Saied, M., Guirguis, S. & Madbouly, M. Review of artificial intelligence for enhancing intrusion detection in the internet of things. *Eng. Appl. Artif. Intell.* **127**, 107231. https://doi.org/10.1016/j.engappai.2023.107231 (2024).
24. Kayode Saheed, Y., Idris Abiodun, A., Misra, S., Holone, M. K. & Colomo-Palacios, R. A machine learning-based intrusion detection for detecting internet of things network attacks. *Alexandria Eng. J.* **61** (12), 9395–9409. https://doi.org/10.1016/j.aej.2022.02.063 (2022).
25. Gaber, T., El-Ghamry, A. & Hassanien, A. E. Injection attack detection using machine learning for smart IoT applications. *Phys. Commun.* **52**, 101685. https://doi.org/10.1016/j.phycom.2022.101685 (2022). September 2020.
26. Rahman, M. A. et al. Effective combining of feature selection techniques for machine learning-enabled IoT intrusion detection. *Multimed Tools Appl.* **80** (20), 31381–31399. https://doi.org/10.1007/s11042-021-10567-y (2021).
27. Shukla, P., Krishna, C. R. & Patil, N. V. Distributed Ensemble Method Using Deep Learning to Detect DDoS Attacks in IoT Networks. *Arab. J. Sci. Eng.* **50** (2), 1143–1168. https://doi.org/10.1007/s13369-024-09144-w (2025).
28. Li, J., Othman, M. S., Chen, H. & Yusuf, L. M. Optimizing IoT intrusion detection system: feature selection versus feature extraction in machine learning. *J. Big Data*. **11** (1). https://doi.org/10.1186/s40537-024-00892-y (2024).
29. Nawaz, M., Tahira, S., Shah, D., Ali, S. & Tahir, M. Lightweight machine learning framework for efficient DDoS attack detection in IoT networks. *Sci. Rep.* **15** (1), 1–24. https://doi.org/10.1038/s41598-025-10092-0 (2025).
30. Saied, M., Guirguis, S. & Madbouly, M. IoT Cybersecurity: On the Use of Boosting-Based Approaches for Botnet Detection. *IT Prof.* **26** (6), 45–54. https://doi.org/10.1109/MITP.2024.3466509 (2024).
31. Saied, M. & Guirguis, S. Explainable artificial intelligence for botnet detection in internet of things. *Sci. Rep.* **15** (1), 1–17. https://doi.org/10.1038/s41598-025-90420-6 (2025).
32. Vajrobol, V. et al. Identify spoofing attacks in Internet of Things (IoT) environments using machine learning algorithms. *J. High. Speed Networks*. **31** (1), 61–70. https://doi.org/10.1177/09266801241295886 (2025).
33. Ismail et al. A Machine Learning-Based Classification and Prediction Technique for DDoS Attacks. *IEEE Access.* **10**, 21443–21454. https://doi.org/10.1109/ACCESS.2022.3152577 (2022).
34. Ahmad, I., Wan, Z. & Ahmad, A. A big data analytics for DDOS attack detection using optimized ensemble framework in Internet of Things. *Internet of Things (Netherlands)*. **23**, (2023). https://doi.org/10.1016/j.iot.2023.100825
35. Albulayhi, K. et al. IoT Intrusion Detection Using Machine Learning with a Novel High Performing Feature Selection Method. *Appl. Sci.* **12** (10). https://doi.org/10.3390/app12105015 (2022).
36. Saied, M., Guirguis, S. & Madbouly, M. A Comparative Study of Using Boosting-Based Machine Learning Algorithms for IoT Network Intrusion Detection. *Int. J. Comput. Intell. Syst.* **16** (1). https://doi.org/10.1007/s44196-023-00355-x (2023).
37. Anish, T. P., Shanmuganathan, C., Dhinakaran, D. & Kumar, V. V. Hybrid Feature Extraction for Analysis of Network System Security—IDS. *Lect Notes Electr. Eng.* **1073 LNEE**, 25–36. https://doi.org/10.1007/978-981-99-5080-5_3 (2023).
38. Asmat, H., Ullah, F., Khan, A. A., Ali, F. & Mohmand, M. I. A Novel caching framework for information-centric IoT using deep reinforcement Proximal Policy Optimization. *Comput. Commun.* **241** https://doi.org/10.1016/j.comcom.2025.108261 (2025).
39. Ullah, F. et al. Lightweight Multimedia Anomaly and Integrity Detection for Consumer IoT Using Knowledge Distillation. *IEEE Trans. Consum. Electron.* https://doi.org/10.1109/TCE.2025.3644297 (2025).
40. Uthansakul, P., Anchuen, P., Uthansakul, M. & Khan, A. A. QoE-Aware self-tuning of service priority factor for resource allocation optimization in LTE networks. *IEEE Trans. Veh. Technol.* **69** (1), 887–900. https://doi.org/10.1109/TVT.2019.2952568 (2020).
41. Gad, A. R., Nashat, A. A. & Barkat, T. M. Intrusion Detection System Using Machine Learning for Vehicular Ad Hoc Networks Based on ToN-IoT Dataset. *IEEE Access.* **9**, 142206–142217. https://doi.org/10.1109/ACCESS.2021.3120626 (2021).
42. Songma, S., Sathuphan, T. & Pamutha, T. Optimizing Intrusion Detection Systems in Three Phases on the CSE-CIC-IDS-2018 Dataset. *Computers* **12** (12). https://doi.org/10.3390/computers12120245 (2023).
43. Neto, E. C. P. et al. CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment. *Sensors* **23** (13). https://doi.org/10.3390/s23135941 (2023).
44. Bokhari, M. U., Khan, M. Z. & Masoodi, F. S. A Hybrid Approach to Feature Selection for Cyber Threat Detection in IoT Networks, in *3rd IEEE International Conference on Device Intelligence, Computing and Communication Technologies, DICCT*, pp. 637–642. https://doi.org/10.1109/DICCT64131.2025.10986689 (2025).
45. Chen, W., Yang, H., Yin, L. & Luo, X. Large-scale IoT attack detection scheme based on LightGBM and feature selection using an improved salp swarm algorithm. *Sci. Rep.* **14** (1), 1–25. https://doi.org/10.1038/s41598-024-69968-2 (2024).
46. Chalichalamala, S., Govindan, N. & Kasarapu, R. Logistic Regression Ensemble Classifier for Intrusion Detection System in Internet of Things. *Sensors* **23** (23). https://doi.org/10.3390/s23239583 (2023).
47. Lin, L. et al. Intrusion detection using a hybrid approach based on CatBoost and an enhanced inception V1. *Int. J. Mach. Learn. Cybern.* **16**, 7–8. https://doi.org/10.1007/s13042-024-02505-9 (2025).
48. Akif, O. Z., Ali, S. M., Sabih, A. F., Sadiq, A. T. & Subramaniam, S. K. Intrusion Detection System for IoT Based on Modified Random Forest Algorithm. *Iraqi J. Comput. Sci. Math.* **6** (2), 221–229. https://doi.org/10.52866/2788-7421.1258 (2025).
49. Chen, R. et al. Incremental Learning of Gradient Boosting Decision Trees to Detect Botnets in Internet of Things. *Sensors* **24** (7), 1–23. https://doi.org/10.3390/s24072083 (2024).
50. Aldaej, A., Ullah, I., Ahanger, T. A. & Atiquzzaman, M. Ensemble technique of intrusion detection for IoT-edge platform. *Sci. Rep.* **14** (1), 1–16. https://doi.org/10.1038/s41598-024-62435-y (2024).
51. Zidan, R. A. & Karraz, G. Towards an Efficient Internet of Things Intrusion Detection by Using Support Vector Machine. *Baghdad Sci. J.* **22** (5), 1714–1724. https://doi.org/10.21123/bsj.2024.11067 (2025).
52. Yang, Y. M., Chang, K. C. & Luo, J. N. Hybrid Neural Network-Based Intrusion Detection System: Leveraging LightGBM and MobileNetV2 for IoT Security. *Symmetry (Basel)*. **17** (3), 1–19. https://doi.org/10.3390/sym17030314 (2025).
53. Ma, H., Zhang, W., Zhang, D. & Chen, B. An IoT intrusion detection framework based on feature selection and large language models fine-tuning. *Sci. Rep.* **15** (1), 1–21. https://doi.org/10.1038/s41598-025-08905-3 (2025).

54. Shambharkar, P. G. & Sharma, N. XAI-driven multi-attention DeepCRNN for enhanced cyberattack detection in internet of medical things environments. *Pervasive Mob. Comput.* **114**, 102123. https://doi.org/10.1016/j.pmcj.2025.102123 (2025).
55. Sharma, N. & Shambharkar, P. G. Transforming security in internet of medical things with advanced deep learning-based intrusion detection frameworks. *Appl. Soft Comput.* **180**, 113420. https://doi.org/10.1016/j.asoc.2025.113420 (2025).
56. Dharini, N., Janani, V. S. & Katiravan, J. Efficient detection of intrusions in TON-IoT dataset using hybrid feature selection approach. *Sci. Rep.* **16** (1), 1–29. https://doi.org/10.1038/s41598-026-37834-y (2026).
57. Kaushik, S. et al. Efficient, Lightweight Cyber Intrusion Detection System for IoT Ecosystems Using MI2G Algorithm. *Computers* **11** (10). https://doi.org/10.3390/computers11100142 (2022).
58. Alotaibi, J. AI-driven intrusion detection and mitigation framework for software-defined IoT networks. *Peer-to-Peer Netw. Appl.* **18** (6). https://doi.org/10.1007/s12083-025-02151-0 (2025).

Acknowledgements
The authors are thankful to the Deanship of Graduate Studies and Scientific Research at University of Bisha for supporting this work through the Fast-Track Research Support Program. Additionally, the authors are also grateful to the Department of Computer Science, Aligarh Muslim University, Aligarh, for facilitating this work.

Author contributions
MZK contributed to the study conception, implementation, design, data acquisition, Writing – original draft preparation, and Writing – review & editing of the model. MA performs conceptualization, methodology, data analysis, Writing – original draft preparation, and Writing – review & editing of the work. MUB, FSM performs formal analysis, data curation, and made under the supervision of the work. ZA performs formal analysis, data curation, and Writing – review & editing. IA and MZK performs formal analysis and Writing – review & editing of the work. All authors are responsible for completing the data analysis, manuscript preparation, and editing. All authors have read and agreed to publish a version of the manuscript.

Declarations

Competing interests
The authors declare no competing interests.

Additional information
Correspondence and requests for materials should be addressed to I.A.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2026